



DAYANANDA SAGAR COLLEGE OF ENGINEERING

(An Autonomous Institute Affiliated to VTU, Belagavi, Accredited by NAAC with 'A' Grade)
Shavige Malleshwara Hills, Kumaraswamy Layout, Bengaluru-560111

Department of Computer Science & Engineering (Cyber Security)

Lab Manual

Devops

22CYL671

Prepared By

Padmavathi S
Assistant Professor
Dept. of CSE(Cyber Security)



DAYANANDA SAGAR COLLEGE OF ENGINEERING

(An Autonomous Institute Affiliated to VTU, Belagavi, Accredited by NAAC with 'A' Grade)
Shavige Malleshwara Hills, Kumaraswamy Layout, Bengaluru-560111

Department of Computer Science & Engineering (Cyber Security)

Ability Enhancement Course - VI (AEC)

DevOps			
Course code: 22CY671	Credits: 01	L: T: P: S: 0:0:2:0	Total Hours: 15
CIE Marks: 50	SEE Marks: 50		Exam Hours: 03

Course Outcomes. At the end of the course, the student will be able to

CO1	Understand the significance of DevOps and its Relationship with Cloud Platforms and Version Control Systems.
CO2	Demonstrate Understanding of the Maven Build Lifecycle
CO3	Implement Continuous integration and Continuous delivery pipelines using Jenkins
CO4	Create and Manage Docker Containers

Sl. No	Programs
1.	Explore Git and GitHub commands
2.	Implement, build, test, configure and monitor the software application in devops(Using flask)
3.	Demonstrate creating job using Jenkins
4.	Configuration Management with Ansible: Basics of Ansible: Inventory, Playbooks and Modules, Automating Server Configurations with Playbooks.
5.	Setting up a Gradle project, understanding build scripts (Groovy), dependency management and task automation
6.	Install and explore Selenium for automated testing.
7.	Develop a simple containerized application using Docker
8.	Automate the process of running containerized application using Kubernetes
9.	Implementing Continuous Security with Snyk
10.	Creating a Maven project , understanding the POM file, dependency management and plugins

Assessment Details (both CIE and SEE)

The weightage of Continuous Internal Evaluation (CIE) is 50% and for Semester End Exam (SEE) is 50%. The minimum passing mark for the CIE is 40% of the maximum marks (20 marks out of 50) and for the SEE minimum passing mark is 35% of the maximum marks (18 out of 50 marks). A student shall be deemed to have satisfied the academic requirements and earned the credits allotted to each subject/ course if the student secures a minimum of 40% (40 marks out of 100) in the sum total of the CIE (Continuous Internal Evaluation) and SEE (Semester End Examination) taken together.

Semester-End Examination:

- SEE marks for the practical course are 50 Marks.
- SEE shall be conducted jointly by the two examiners of the same institute, examiners are appointed by the Head of the Institute.
- All laboratory experiments are to be included for practical examination.
- Students can pick one question (experiment) from the questions lot prepared by the examiners jointly.

- Evaluation of test write-up/ conduction procedure and result/viva will be conducted jointly by examiners.
- General rubrics suggested for SEE are mentioned here, writeup-20%, Conduction procedure and result in -60%, Viva-voce 20% of maximum marks.
- Change of experiment is allowed only once and 15% of Marks allotted to the procedure part are to be made zero.
- The minimum duration of SEE is 03 hours

Continuous Internal Evaluation: for the practical component

On completion of every experiment/program in the laboratory, the students shall be evaluated and marks shall, be awarded on the same day. The 30 marks are for conducting the experiment and preparation of the laboratory record is for 10 marks, the other 10 marks shall be for the test conducted at the end of the semester.

Typical Evaluation pattern for AEC course is shown in Table.

Component		Max Marks	Total Marks	Min Marks
CIE Lab	Practical(Conduction)	30	50	12
	Practical(Record)	10		-
	Practical(Test)	10		8
Total CIE Practical			50	20
SEE	Semester End Examination	50	50	18
CIE+SEE			100	40

Appendix

Pre-requisites and Common Setup

Before starting any experiment, ensure you have the following installed and configured on your Ubuntu system:

1. Java Development Kit (JDK):

Install OpenJDK (version 11 or above):

```
sudo apt update  
sudo apt install openjdk-11-jdk
```

Verify installation:

```
java -version
```

2. Git (for version control):

```
sudo apt install git
```

Configure Git (replace with your details):

```
git config --global user.name "Your Name"  
git config --global user.email "your.email@example.com"
```

3. Maven:

```
sudo apt install maven
```

Verify installation:

```
mvn -version
```

4. Gradle:

```
sudo apt install gradle
```

Verify installation:

```
gradle -v
```

5. Jenkins:

Follow these steps to install Jenkins:

```
wget -q -O - https://pkg.jenkins.io/debian-stable/jenkins.io.key |  
sudo apt-key add -  
sudo sh -c 'echo deb https://pkg.jenkins.io/debian-stable binary/ >  
/etc/apt/sources.list.d/jenkins.list'  
sudo apt update  
sudo apt install jenkins  
sudo systemctl start jenkins  
sudo systemctl status jenkins
```

Open Jenkins in your browser at <http://localhost:8080>.

6. Ansible:

```
sudo apt update  
sudo apt install ansible
```

Verify installation:

```
ansible --version
```

Understanding DevOps

1. What is DevOps?

DevOps is a set of cultural philosophies, practices, and tools that combine software development (Dev) and IT operations (Ops). Its goal is to shorten the systems development life cycle while delivering features, fixes, and updates frequently in close alignment with business objectives. DevOps promotes:

- **Collaboration:** Breaking down silos between development and operations teams.
- **Automation:** Automating repetitive tasks (builds, tests, deployments) to improve efficiency.
- **Continuous Integration/Delivery (CI/CD):** Regularly integrating code changes and deploying them to production with minimal manual intervention.
- **Monitoring and Feedback:** Constantly monitoring the performance and behavior of applications in production to rapidly address issues.

Why is DevOps Used?

- **Speed and Agility:** Faster development cycles and quicker time-to-market.
- **Quality:** Automated testing and integration help catch issues early.
- **Reliability:** Frequent, smaller updates reduce the risk of large-scale failures.
- **Efficiency:** Automation reduces manual errors and repetitive tasks.
- **Scalability:** Processes and infrastructure can grow with the business.

Examples of DevOps in Action:

- **Continuous Integration (CI):** Tools like Jenkins or Azure Pipelines automatically build and test code on every commit.
- **Continuous Deployment (CD):** Systems automatically deploy tested code to production environments.
- **Monitoring:** Tools such as Prometheus, Grafana, or New Relic continuously monitor application performance.

2. Introduction to Maven and Gradle

What is Maven?

Maven is a build automation and project management tool primarily used for Java projects. It uses a central configuration file known as the **POM (Project Object Model)**, written in XML, which defines the project structure, its dependencies, build order, and plugins.

Key Features and Roles of Maven:

- **Convention over Configuration:** Maven enforces a standard directory structure (e.g., `src/main/java`, `src/test/java`), which simplifies project setup.
- **Dependency Management:** Automatically downloads and manages external libraries and dependencies from repositories like Maven Central.
- **Build Lifecycle:** Defines a fixed lifecycle (e.g., compile, test, package, install, deploy) which standardizes the build process.
- **Plugin Ecosystem:** Provides a rich set of plugins to extend functionality (e.g., unit testing, code coverage, reporting).

3. What is Gradle?

Gradle is a modern build automation tool that is known for its flexibility and performance. It uses a **Groovy** or **Kotlin DSL (Domain Specific Language)** to define build logic, which allows for more dynamic and customizable configurations compared to Maven's XML-based approach.

Key Features and Roles of Gradle:

- **Flexible Build Scripts:** Instead of a rigid XML file, Gradle build scripts written in Groovy or Kotlin allow you to include logic, conditionals, and loops.
- **Incremental Builds:** Gradle tracks changes in source files and only rebuilds what is necessary, which can significantly speed up the build process.
- **Multi-project Builds:** Easily manages complex projects with multiple modules or subprojects.
- **Extensibility:** A robust plugin system that enables you to integrate various languages, frameworks, and tools.
- **Parallel Execution:** Can run tasks in parallel, optimizing build times for large projects.

4. Why Use Maven and Gradle?

Both Maven and Gradle are used to automate the build process and manage project

dependencies. Here's why they are so popular in the DevOps ecosystem:

- **Automated Build and Testing:** They allow developers to compile code, run tests, and package applications without manual intervention.
- **Consistent Build Environment:** By enforcing standardized project structures and dependency management, they help avoid the "it works on my machine" problem.
- **Ease of Integration:** Both tools integrate well with Continuous Integration (CI) servers like Jenkins, enabling automated pipelines.
- **Dependency Resolution:** They simplify the process of managing external libraries and ensure that all developers are using the same versions.

5. Installing Maven and Gradle on Ubuntu

A. Installing Maven

Step 1: Update Your System

Open a terminal and run:

```
sudo apt update
sudo apt upgrade
```

Step 2: Install Java (if not already installed)

Maven requires Java. Install OpenJDK (for example, version 11):

```
sudo apt install openjdk-11-jdk
java -version
```

Expected Output (example):

```
openjdk version "11.0.11" 2021-04-20
OpenJDK Runtime Environment (build 11.0.11+9-Ubuntu-0ubuntu2.20.04)
OpenJDK 64-Bit Server VM (build 11.0.11+9-Ubuntu-0ubuntu2.20.04, mixed mode,
sharing)
```

Step 3: Install Maven

Run the following command:

```
sudo apt install maven
```


After installation, check the Maven version:

```
mvn -version
```

Expected Output (example):

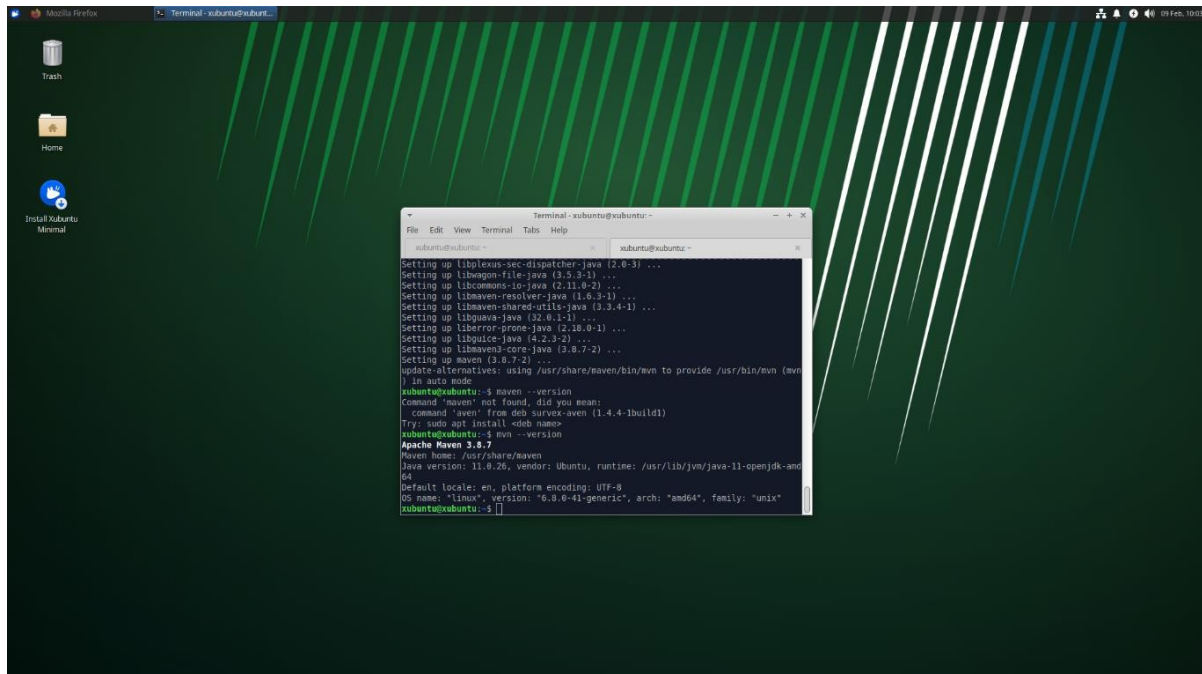
Apache Maven 3.6.3

Maven home: /usr/share/maven

Java version: 11.0.11, vendor: Ubuntu, runtime: /usr/lib/jvm/java-11-openjdk-amd64

Default locale: en_US, platform encoding: UTF-8

OS name: "linux", version: "5.4.0-xx-generic", arch: "amd64", family: "unix"



Installing Gradle

Gradle can be installed in two primary ways: via the Ubuntu repositories (which may not be the latest version) or by manually installing the latest version.

Option 1: Install from Ubuntu Repositories

```
sudo apt install gradle
```

```
gradle -v
```

Screenshot Tip: Capture the terminal output of `gradle -v`.

Option 2: Install the Latest Version Manually (Recommended)

1. Download Gradle:

```
wget https://services.gradle.org/distributions/gradle-8.0-bin.zip
```

2. Unzip the Downloaded File:

```
sudo unzip -d /opt/gradle gradle-8.0-bin.zip
```

3. Set Up the Environment Variables: Add Gradle to your system PATH by appending the following line to your `~/.bashrc` (or `~/.profile`):

```
echo "export PATH=\$PATH:/opt/gradle/gradle-8.0/bin" >> ~/.bashrc
source ~/.bashrc
```

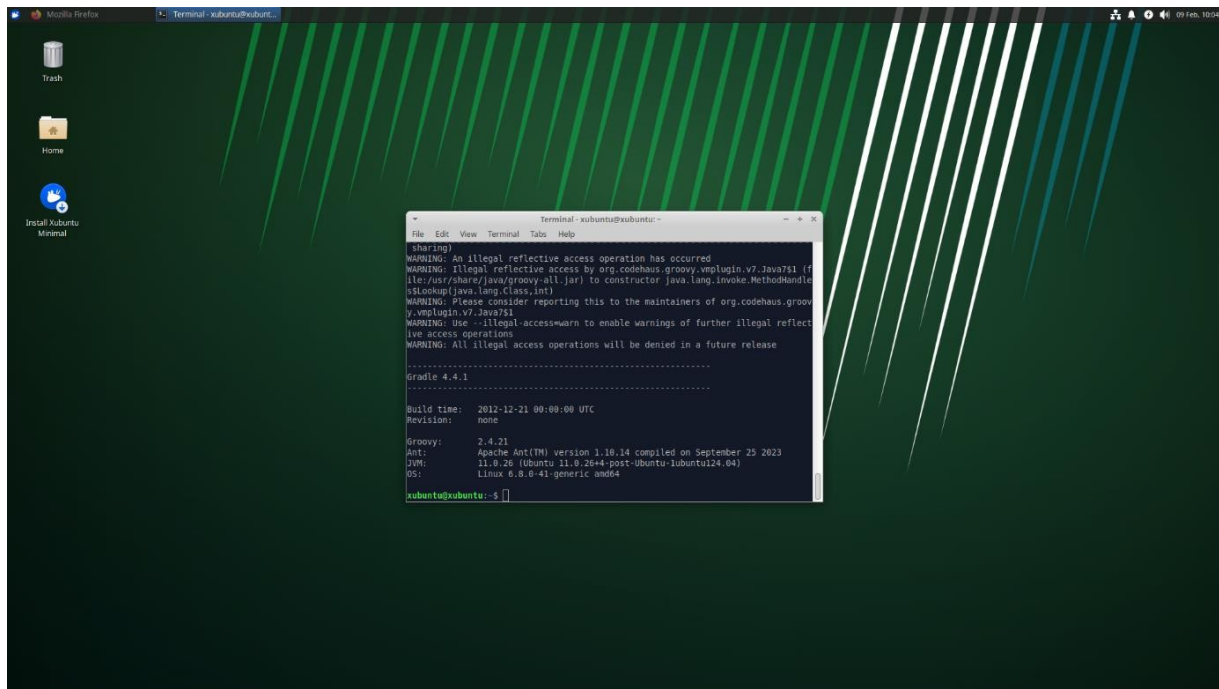
4. Verify the Installation:

```
gradle -v
```

Expected Output (example):

Gradle 8.0

```
Build time:    2022-05-03 12:00:00 UTC
Revision:     <revision-hash>
Kotlin:       1.6.x
Groovy:       3.0.x
Ant:          Apache Ant(TM) version 1.10.x compiled on ...
JVM:          11.0.11 (Ubuntu 11.0.11+9-Ubuntu-0ubuntu2.20.04)
OS:           Linux 5.4.0-xx-generic amd64
```



6. Detailed Differences Between Maven and Gradle

Below is a detailed comparison highlighting their main differences

Aspect	Maven	Gradle
Build Script Language	Uses an XML-based configuration file (pom.xml).	Uses a DSL based on Groovy or Kotlin (build.gradle or build.gradle.kts).
Configuration Style	Declarative and rigid – follows strict conventions (convention over configuration).	Flexible and dynamic – allows you to write custom logic and conditions within the build script.
Build Lifecycle	Provides a fixed lifecycle (e.g., validate, compile, test, package, install, deploy).	Uses a task-based approach where tasks can be defined, customized, and linked in a flexible manner.
Dependency Management	Manages dependencies through the POM file; downloads them from central repositories (e.g., Maven Central).	Similar dependency management; supports dynamic version resolution and customizable dependency configurations.

Performance	Generally slower for large projects because of the fixed build lifecycle and less emphasis on incremental builds.	Often faster, thanks to incremental builds, caching, and parallel execution of tasks.
Extensibility & Plugins	Rich ecosystem of plugins but customization can be more challenging due to XML's verbosity and limitations in scripting.	Highly extensible through its scripting capabilities; writing custom tasks or plugins in Groovy/Kotlin is more straightforward.
Multi-Project Builds	Handles multi-module projects well but requires a strict directory layout and a parent POM for aggregating modules.	Excels in multi-project builds with simple configuration, allowing each subproject to be configured in a flexible manner.
Learning Curve	Easier for beginners due to its structured, convention-based approach but can become complex with large configurations.	More flexible but may have a steeper learning curve initially if you need to leverage its dynamic features and custom logic.
Community & Maturity	Has been around longer, so many legacy projects and extensive documentation exist.	Relatively newer; it has gained popularity due to its modern features, especially in Android and multi-language projects.
Integration with CI/CD	Well-supported by most CI/CD tools (like Jenkins, Azure Pipelines) with stable, predictable behavior.	Also integrates seamlessly with CI/CD tools and is often chosen for its faster build times and flexibility in pipeline scripting.

Experiment 1: Explore Git and GitHub commands

What is Git?

Git is a **version control system (VCS)** that lets you manage and track changes to your code or files over time. It was developed by **Linus Torvalds**, the creator of Linux, and is widely used in software development.

Key Features of Git:

- **Tracks changes:** Helps in keeping a history of file modifications.
- **Branching and merging:** You can work on different features in parallel and combine them later.
- **Distributed system:** Every developer has the full code history locally.
- **Speed and efficiency:** Very fast performance compared to older VCS like SVN.

Example Use Cases:

- Keeping track of a coding project.
- Collaborating with others without overwriting each other's work.
- Reverting to an earlier version if something breaks.

What is GitHub?

GitHub is a **cloud-based platform** built around Git. It hosts your Git repositories online and provides tools for collaboration, version control, and project management.

Key Features of GitHub:

- **Remote repositories:** Store your Git repositories online.
 - **Collaboration:** Multiple users can work on the same project.
 - **Pull requests:** Suggest changes and get them reviewed before merging.
 - **Issue tracking:** Manage bugs and feature requests.
 - **Actions:** Automate workflows like testing and deployment.
-

Difference Between Git and GitHub:

Feature	Git	GitHub
Type	Version Control System	Cloud-based Hosting Service
Use	Track changes locally	Share and collaborate remotely
Internet Required	No	Yes
Main Function	Code management	Project collaboration and hosting

1. git init

Initializes a new Git repository in the current directory.

2. git config

Sets configuration values like username and email.

Example: `git config --global user.name "Your Name"`

3. git status

Displays the state of the working directory and staging area.

4. git add <file>

Adds the specified file(s) to the staging area.

5. git commit -m "message"

Records the staged changes to the repository with a message.

6. git log

Shows the commit history for the current branch.

7. git branch

Lists all local branches in the repository.

8. git branch <branch-name>

Creates a new branch.

9. git checkout <branch-name>

Switches to the specified branch.

10. git merge <branch-name>

Merges the specified branch into the current branch.

11. git remote add origin <url>

Adds a remote repository URL to your local repo.

12. git push -u origin <branch-name>

Pushes the branch to the remote repository and sets the upstream.

13. git pull

Fetches and integrates changes from the remote repository into the current branch.

14. git clone <url>

Creates a local copy of a remote repository.

15. git diff

Shows the differences between files in the working directory and the staging area.

16. git reset <file>

Unstages the file, keeping the changes in the working directory.

17. git rm <file>

Removes the file from the working directory and stages the removal.

18. git stash

Temporarily stashes changes that are not ready to be committed.

19. git stash pop

Applies the last stashed changes and removes them from the stash list.

20. git fetch

Downloads objects and refs from another repository.

Experiment 2: Implement, build, test, configure and monitor the software application in Devops (Using flask)

What is Flask?

Flask is a **lightweight, open-source web framework** written in **Python**. It is used to build web applications quickly and with minimal code. Flask is known for its simplicity, flexibility, and ease of use.

It is called a "**micro**" **framework** because it does not come with built-in tools like database integration or form validation—but you can add them as needed.

- Implement phase refers to the actual coding or programming of the application by developers.
- The build phase involves converting the source code into an executable form that can be deployed.
- Testing ensures that the application behaves as expected and is bug-free before going live.
- Configuration is the process of preparing the environment where the app will run, including tools, servers, and files.
- Monitoring is essential for keeping track of system health, performance, and user activity after deployment.

1. Install Prerequisites

Update system packages and install required software:

```
sudo apt update && sudo apt upgrade
```

```
sudo apt install python3 python3-pip git virtualenv -y
```

2. Create and Set Up Flask Application

```
mkdir flask_app
```

```
cd flask_app
```

```
python3 -m venv venv  
source venv/bin/activate  
pip install flask
```

Step 1: Create 'app.py' with the following content:

```
from flask import Flask  
app = Flask(__name__)  
@app.route('/')  
def home():  
    return "Hello, DevOps!"  
if __name__ == "__main__":  
    app.run(host='0.0.0.0', port=5000)
```

Step 2: Build

If you know, Python is an interpreted language. There is no build step so to speak, since the code gets executed line by line. So here, build refers to the preparation of the environment and making sure the app is ready to run.

```
requirements.txt  
flask = 2.1.0
```

We will write a simple shell script (**build.sh**) to install dependencies and ensure the app runs:

```
#!/bin/bash  
echo "Setting up environment..."  
python3 -m venv venv  
source venv/bin/activate  
pip install -r requirements.txt  
echo "Environment setup complete."
```

Run the app with: `python app.py`

Run the script:

`bash build.sh`

Step 3: Test

Now since the program demands we write some test, we will write a very simple test that checks if the app is running or not. Btw shameless plugin but if you want to write tests with 0 code, checkout my Keploy Vscode Extension xD

Test File (`test_app.py`):

```
import unittest

from app import app

class TestApp(unittest.TestCase):

    def test_home(self):

        tester = app.test_client(self)

        response = tester.get('/')

        self.assertEqual(response.status_code, 200)

        self.assertEqual(response.data, b"Hello, DevOps!")

if __name__ == "__main__":

    unittest.main()
```

Run the tests:

`python -m unittest test_app.py`

Step 4: Configure

Configuration here focuses on making sure the app can run in different environments. One simple approach is to use environment variables.

Create a .env file:

```
FLASK_PORT=5000
```

Modify app.py to read the environment variable:

```
import os

from flask import Flask

app = Flask(__name__)

@app.route('/')

def home():

    return "Hello, DevOps!"

if __name__ == "__main__":

    port = int(os.getenv("FLASK_PORT", 5000))

    app.run(port=port)
```

Use python-dotenv to automatically load .env files:

pip install python-dotenv

Add to app.py:

```
from dotenv import load_dotenv

load_dotenv()
```

Step 5: Monitor

Monitoring in general means just seeing what the app is doing while it is running , like logs, health , any errors etc. We will integrate some basic monitoring without external tools:

Add a /health endpoint to check if the app is running:

```
@app.route('/health')

def health():
```

```
return {"status": "up"}, 200
```

Check the output in localhost : 5000/health path.

Experiment 3: Demonstrate creating job using Jenkins

1. What Is Jenkins?

Definition and Overview

Jenkins is an open-source automation server written in Java. It is widely used to facilitate **Continuous Integration (CI)** and **Continuous Deployment/Delivery (CD)** pipelines in software development. Jenkins automates repetitive tasks related to building, testing, and deploying software, helping teams to integrate changes continuously and deliver high-quality applications faster.

Key Functionalities

- **Continuous Integration/Delivery:** Automatically builds, tests, and deploys code after each commit.
- **Extensible Plugin Ecosystem:** Over 1,500 plugins allow integration with numerous tools such as Git, Maven, Gradle, Docker, and many cloud providers.
- **Automated Builds:** Jenkins can poll version control systems, trigger builds on code changes, and even schedule builds.
- **Pipeline as Code:** With the introduction of Jenkins Pipeline (using either scripted or declarative syntax), you can define your entire build process in a code file (Jenkinsfile) and store it alongside your code.
- **Distributed Builds:** Jenkins can distribute workloads across multiple machines, helping to run tests and builds faster.
- **Monitoring and Reporting:** Provides detailed logs, test reports, and build history.

Why Use Jenkins?

- **Improved Efficiency:** Automates mundane tasks to free up developer time.
 - **Rapid Feedback:** Quickly identifies integration issues with automated tests.
 - **Scalability:** Supports distributed builds and parallel testing.
 - **Customization:** A highly configurable system that can integrate with nearly every tool in the software development lifecycle.
-

- **Community Support:** Extensive community and plugin ecosystem that continuously evolve Jenkins' capabilities.

2. Pre-requisites

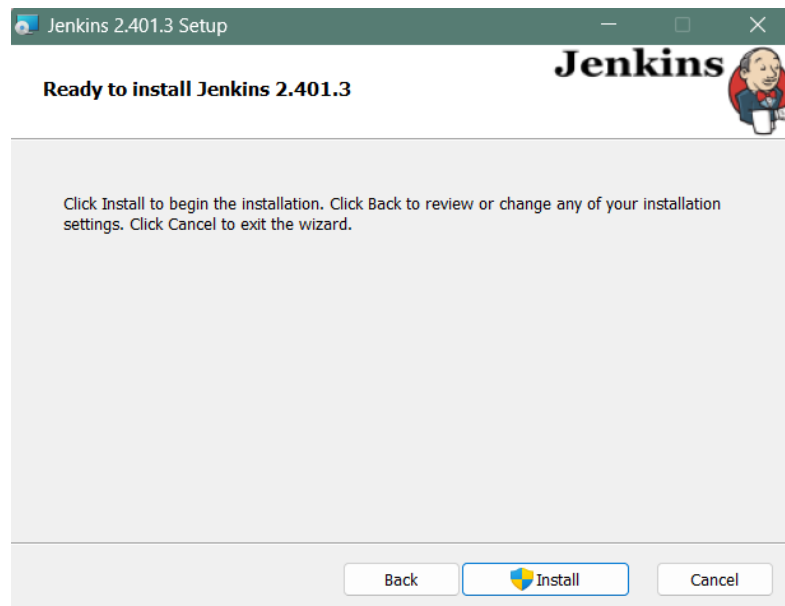
Minimum hardware requirements:

4 GB+ of RAM

50 GB+ of drive space

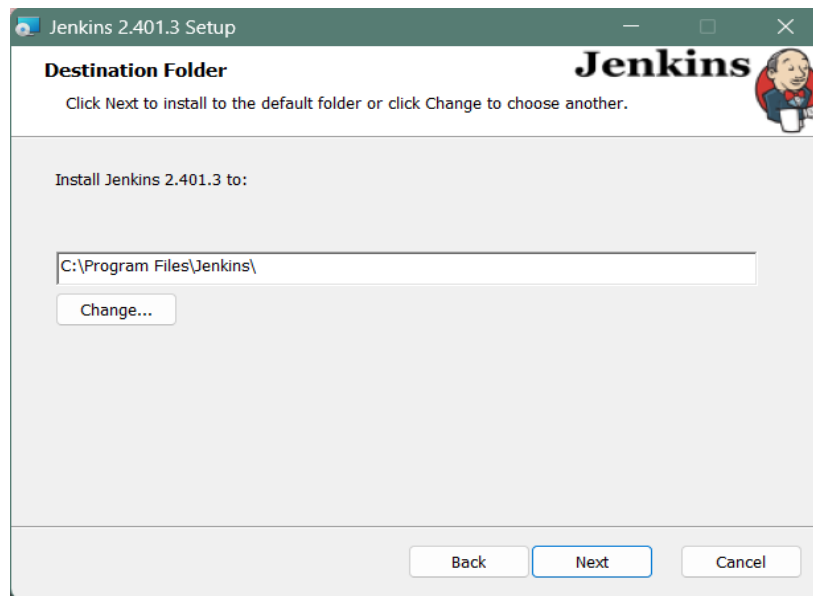
Step 1: Setup wizard

On opening the Windows Installer, an Installation Setup Wizard appears, Click Next on the Setup Wizard to start your installation.



Step 2: Select destination folder

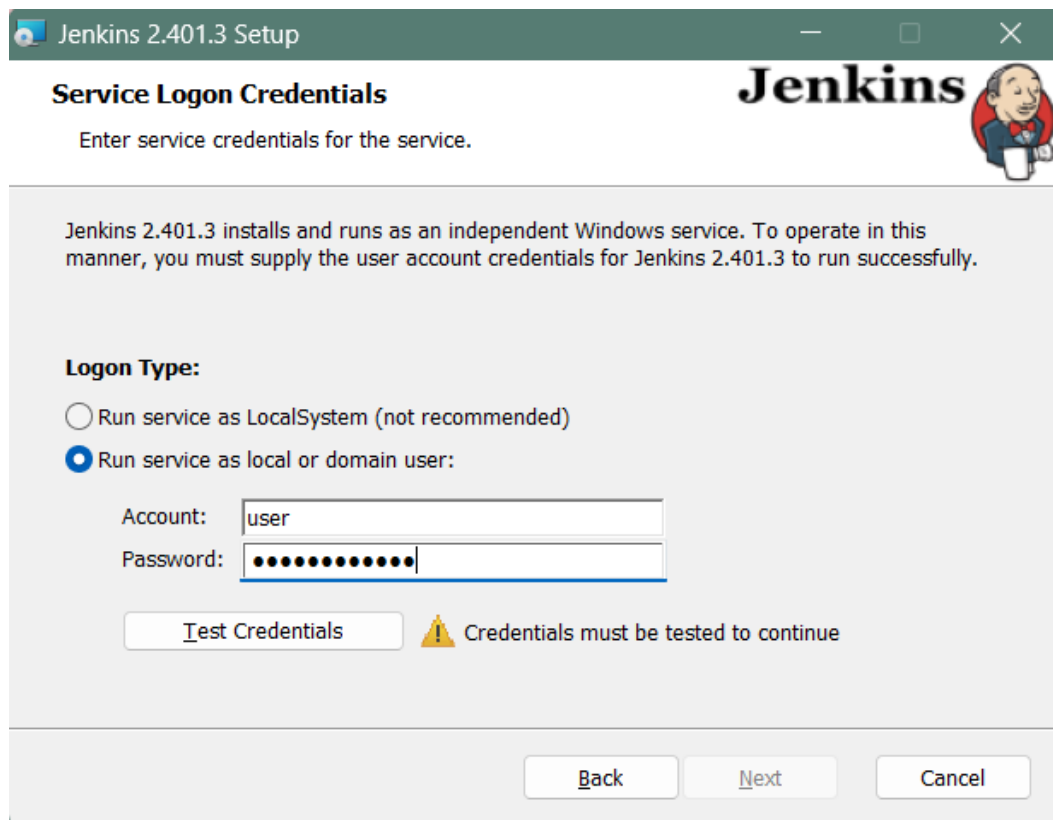
Select the destination folder to store your Jenkins Installation and click Next to continue.



Step 3: Service logon credentials

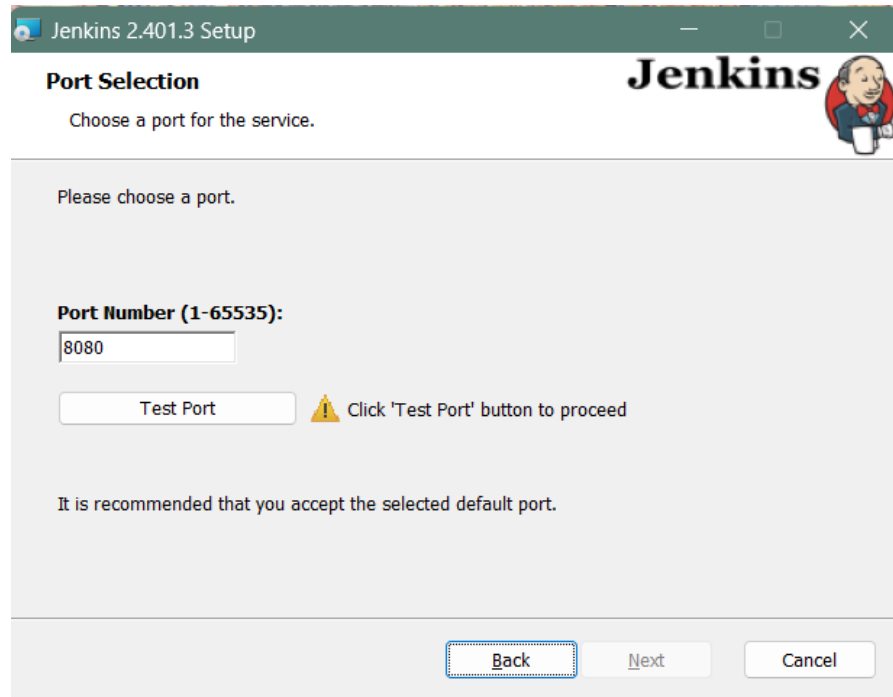
When Installing Jenkins, it is recommended to install and run Jenkins as an independent windows service using a local or domain user as it is much safer than running Jenkins using LocalSystem(Windows equivalent of root) which will grant Jenkins full access to your machine and services.

To run Jenkins service using a local or domain user, specify the domain user name and password with which you want to run Jenkins, click on Test Credentials to test your domain credentials and click on Next.



Step 4: Port selection

Specify the port on which Jenkins will be running, Test Port button to validate whether the specified port is free on your machine or not. Consequently, if the port is free, it will show a green tick mark as shown below, then click on Next.

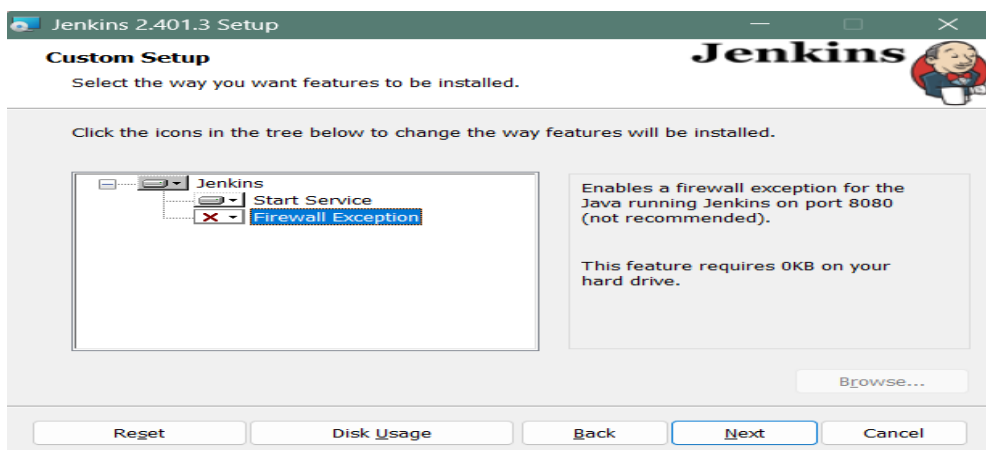


Step 5: Select Java home directory

The installation process checks for Java on your machine and prefills the dialog with the Java home directory. If the needed Java version is not installed on your machine, you will be prompted to install it. Once your Java home directory has been selected, click on Next to continue.

Step 6: Custom setup

Select other services that need to be installed with Jenkins and click on Next.

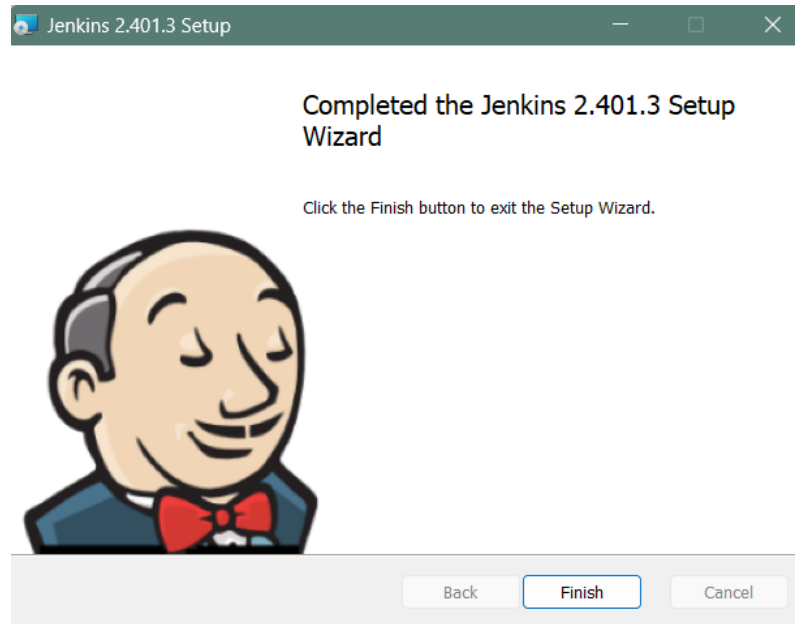


Step 7: Install Jenkins

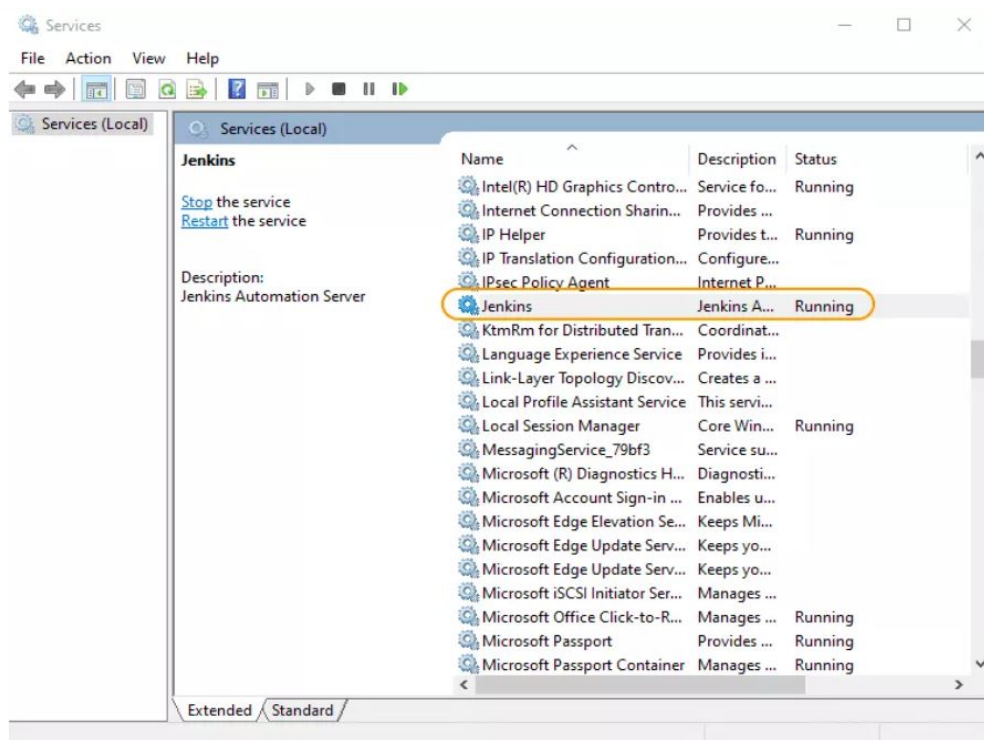
Click on the Install button to start the installation of Jenkins.

Step 8: Finish Jenkins installation

Once the installation completes, click on Finish to complete the installation.



Jenkins will be installed as a Windows Service. You can validate this by browsing the services section, as shown below:



Post-installation setup wizard

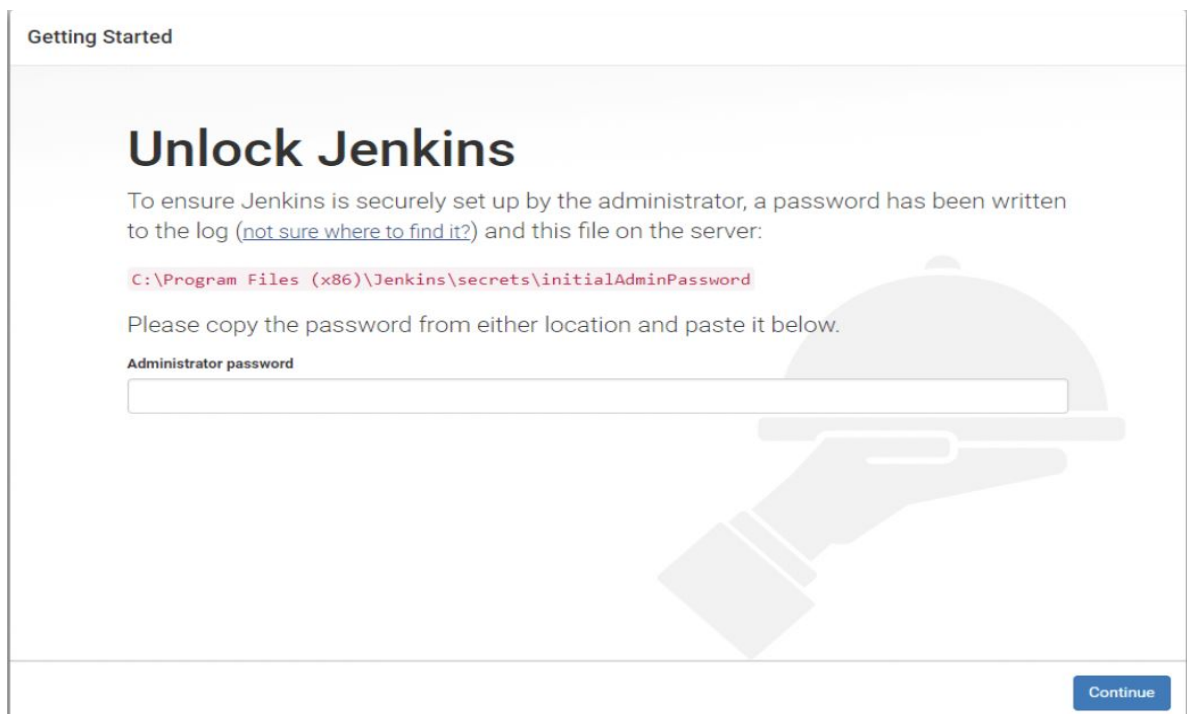
After downloading, installing and running Jenkins, the post-installation setup wizard begins.

This setup wizard takes you through a few quick "one-off" steps to unlock Jenkins, customize it with plugins and create the first administrator user through which you can continue accessing Jenkins.

Unlocking Jenkins

Step 1

Browse to `http://localhost:8080` (or whichever port you configured for Jenkins when installing it) and wait until the **Unlock Jenkins** page appears. When you first access a new Jenkins controller, you are asked to unlock it using an automatically-generated password. (This password will be in `jenkins.err.log` file)



Getting Started

Unlock Jenkins

To ensure Jenkins is securely set up by the administrator, a password has been written to the log ([not sure where to find it?](#)) and this file on the server:

`C:\Program Files (x86)\Jenkins\secrets\initialAdminPassword`

Please copy the password from either location and paste it below.

Administrator password

Continue

Customizing Jenkins with plugins

After unlocking Jenkins, the **Customize Jenkins** page appears. Here you can install any number of useful plugins as part of your initial setup.

Click one of the two options shown:

Install suggested plugins - to install the recommended set of plugins, which are based on most common use cases.

Creating the first administrator user

Finally, after customizing Jenkins with plugins, Jenkins asks you to create your first administrator user. When the **Create First Admin User** page appears, specify the details for your administrator user in the respective fields and click **Save and Finish**. When the **Jenkins is ready** page appears, click **Start using Jenkins**.

Notes:

This page may indicate **Jenkins is almost ready!** instead and if so, click **Restart**.

If the page does not automatically refresh after a minute, use your web browser to refresh the page manually. If required, log in to Jenkins with the credentials of the user you just created and you are ready to start using Jenkins!

Start Using Jenkins

Once the setup is complete, you will be taken to the Jenkins dashboard where you can:

- Create new jobs (Freestyle projects, Pipelines, etc.)
- Install additional plugins as needed
- Monitor builds and view logs.

Experiment 4: Configuration Management with Ansible: Basics of Ansible: Inventory, Playbooks and Modules, Automating Server Configurations with Playbooks.

What Is Ansible?

Ansible is an open-source IT automation and configuration management tool. It allows you to manage multiple servers and perform tasks such as:

- **Configuration Management:** Automate the configuration of servers.
- **Application Deployment:** Deploy applications consistently.
- **Orchestration:** Coordinate complex IT workflows and processes.

Key Concepts in Ansible

- **Inventory:**
An inventory is a file (usually in INI or YAML format) that lists the hosts (or groups of hosts) you want to manage. It tells Ansible which machines to target.
- **Playbook:**
A playbook is a YAML file that defines a set of tasks to be executed on your target hosts. It is the heart of Ansible automation. In a playbook, you specify:
 - **Hosts:** The target machines (or groups) on which the tasks should run.
 - **Tasks:** A list of actions (using modules) that should be executed.
 - **Modules:** Reusable, standalone scripts that perform specific actions (e.g., installing packages, copying files, configuring services).
- **Modules:**
Ansible comes with a large collection of built-in modules (such as `apt`, `yum`, `copy`, `service`, etc.). These modules perform specific tasks on target hosts. You can also write custom modules if needed.

Why Use Ansible?

- **Agentless:** Ansible uses SSH to communicate with target hosts, so no agent needs to be installed on them.
 - **Simplicity:** Playbooks use simple YAML syntax, making them easy to write and understand.
 - **Idempotence:** Ansible tasks are idempotent, meaning running the same playbook multiple times yields the same result, ensuring consistency.
 - **Scalability:** Ansible can manage a small number of servers to large infrastructures with
-

hundreds or thousands of nodes.

2. Installing Ansible on Ubuntu

Before writing a playbook, you need to install Ansible on your control machine (your local Ubuntu system).

Step 1: Update Your System

Open your terminal and run:

```
sudo apt update
sudo apt upgrade -y
```

Step 2: Install Ansible

Install Ansible using apt:

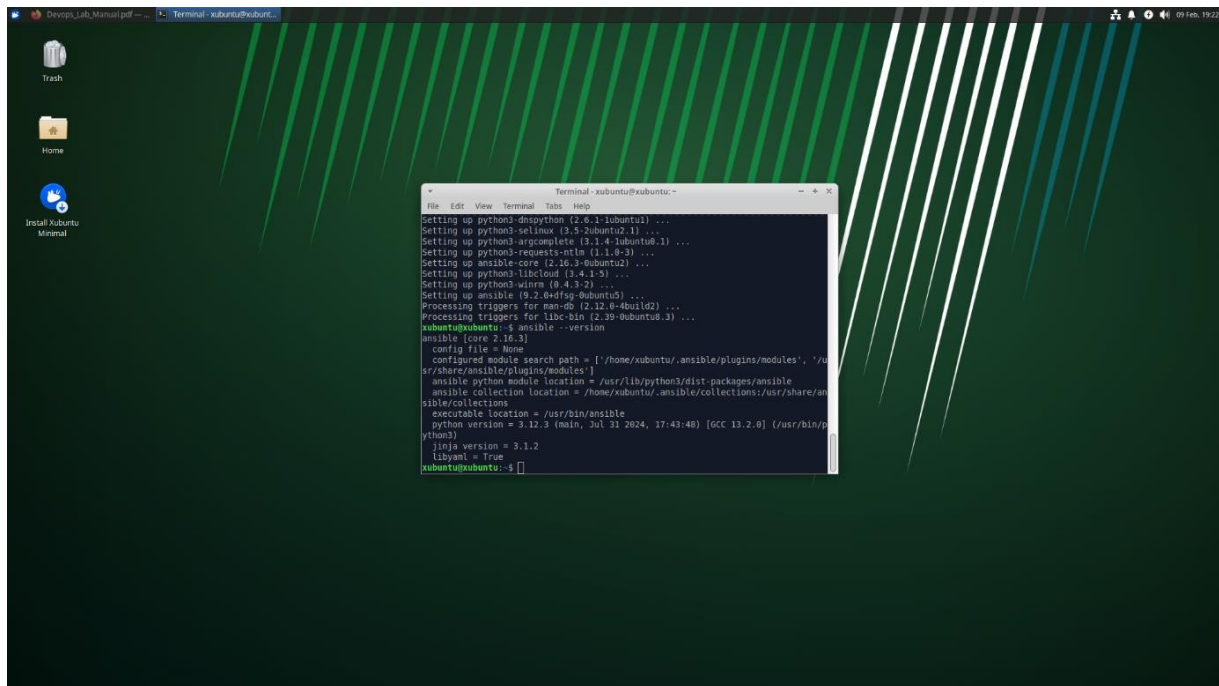
```
sudo apt install ansible -y
```

Verify the installation by checking the version:

```
ansible --version
```

Expected Output Example:

```
ansible 2.9.x
config file = /etc/ansible/ansible.cfg
...
```



3. Creating an Ansible Inventory

An inventory file lists the hosts you want to manage. For this experiment, you can use the local host.

Step 1: Create an Inventory File

1. Open your text editor to create a file called `hosts.ini`:

1. `nano hosts.ini`

1. Add the following content to define the local host:

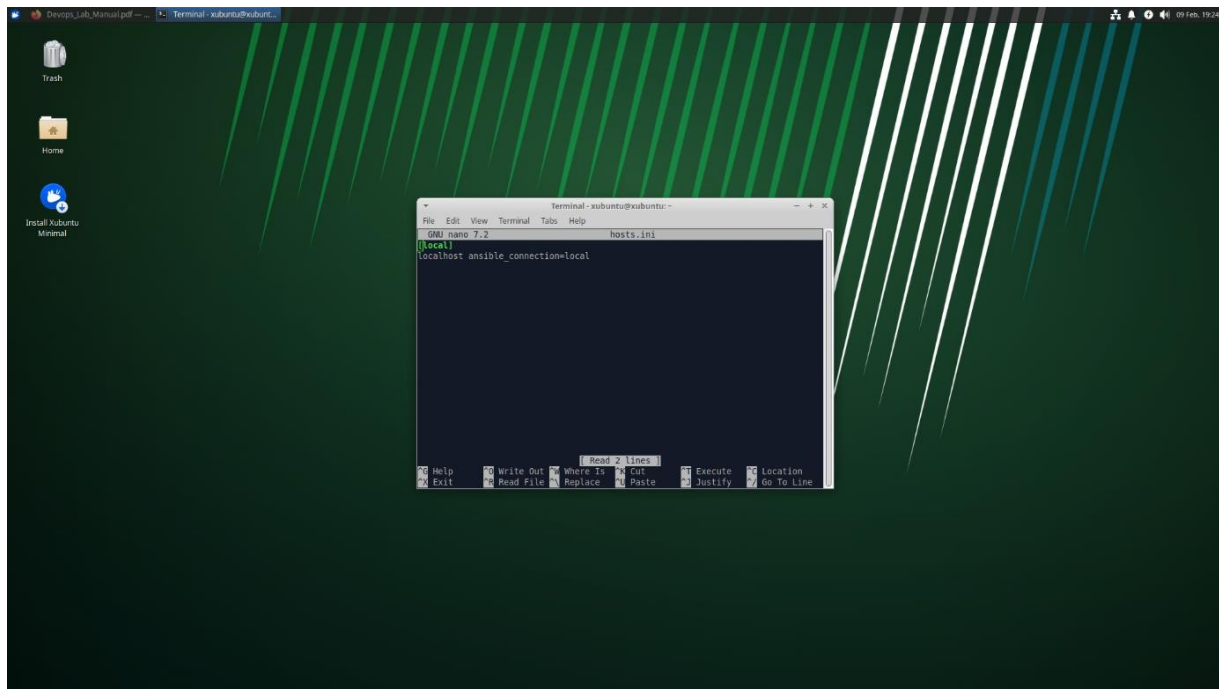
1. `[local]`

2. `localhost ansible_connection=local`

- o **Explanation:**

- `[local]` is a group name.
- `localhost` is the target host.
- `ansible_connection=local` tells Ansible to execute commands on the local machine without SSH.

3. Save the file by pressing **Ctrl+O** then **Enter**, and exit with **Ctrl+X**.



2. Automated Server Configurations

While our experiment covered the basics, here's how you can extend it:

- **Configuring Services:**
Use modules like `service` to start, stop, or restart services. For example, you can automate the configuration of web servers (e.g., Apache or Nginx).
- **Managing Files and Templates:**
Use the `copy` or `template` modules to deploy configuration files across your servers. This is useful for maintaining consistent configuration settings.
- **User and Group Management:** The `user` and `group` modules allow you to create or modify user accounts, ensuring that the correct permissions and roles are applied automatically.
- **Advanced Orchestration:**
Ansible playbooks can include conditionals, loops, and error handling to manage more complex setups, ensuring idempotence and consistency across your infrastructure.

Example: Automating a Web Server Configuration

If you wanted to automate the configuration of an Nginx server, your playbook might include tasks such as:

```
- name: Configure Nginx Web Server
  hosts: webservers
  become: yes
  tasks:
    - name: Update apt cache
      apt:
        update_cache: yes

    - name: Install Nginx
      apt:
        name: nginx
        state: present

    - name: Copy Nginx configuration file
      template:
        src: nginx.conf.j2
        dest: /etc/nginx/nginx.conf
      notify:
        - Restart Nginx

  handlers:
    - name: Restart Nginx
      service:
        name: nginx
        state: restarted
```

5. Writing a Basic Ansible Playbook

You will now create a simple playbook that performs two common tasks:

- **Updating the apt cache**
- **Installing a package (e.g., curl)**

Step 1: Create the Playbook File

1. Open your text editor to create a file called `setup.yml`:
 2. `nano setup.yml`
-

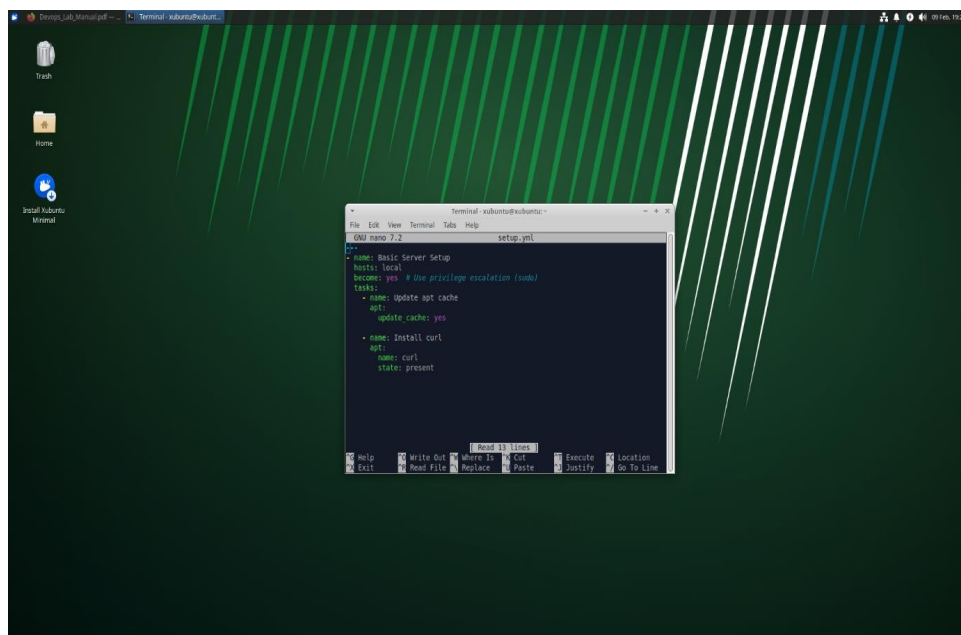
3. Add the following YAML content:

```
1. ---
2. - name: Basic Server Setup
3.   hosts: local
4.   become: yes # Use privilege escalation (sudo)
5.   tasks:
6.     - name: Update apt cache
7.       apt:
8.         update_cache: yes
9.
10.    - name: Install curl
11.      apt:
12.        name: curl
13.        state: present
```

o Explanation:

- **name:** Provides a descriptive name for the play.
- **hosts:** Specifies the group or hosts from the inventory file.
- **become: yes:** Uses `sudo` to perform tasks that require elevated privileges.
- **Tasks Section:**
 - **Update apt cache:** Uses the `apt` module to update the package cache.
 - **Install curl:** Uses the `apt` module to install the `curl` package if it isn't already installed.

14. Save and exit the file.



6. Running the Ansible Playbook

Step 1: Execute the Playbook

In your terminal, run the following command:

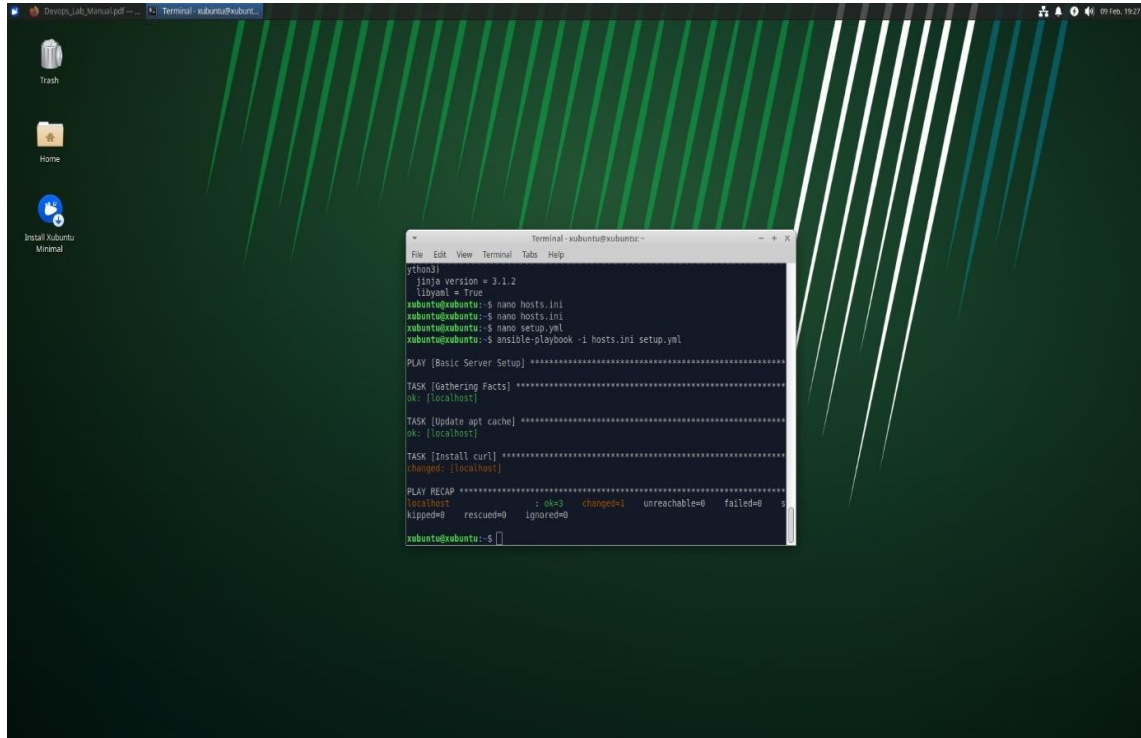
```
ansible-playbook -i hosts.ini setup.yml
```

- **Explanation:**

- `ansible-playbook`: The command to run an Ansible playbook.
- `-i hosts.ini`: Specifies the inventory file.
- `setup.yml`: The playbook file you just created.

Expected Output:

- An output that details each task:
 - For example, a task summary might show “ok=2 changed=1” (if the apt cache was updated and curl was installed).

A screenshot of a Linux desktop environment with a dark green background and diagonal stripes. A terminal window titled 'Terminal - xubuntu@xubuntu:~' is open in the foreground. The terminal shows the execution of an Ansible playbook. The output includes the Jinja version, library path, and the execution of the 'setup.yml' playbook. The playbook consists of three tasks: 'Gathering Facts', 'Update apt cache', and 'Install curl'. The 'Gathering Facts' task is successful on the localhost. The 'Update apt cache' task is also successful on the localhost. The 'Install curl' task is successful on the localhost, with the output 'changed: [localhost]'. The final output shows a summary of the playbook execution: 'PLAY RECAP' with 'localhost' having 'ok=3', 'changed=1', 'unreachable=0', and 'failed=0'. The terminal prompt is 'xubuntu@xubuntu: ~\$'.

Experiment 5: Setting up a Gradle project, understanding build scripts (Groovy), dependency management and task automation

1. Introduction to Gradle

Gradle is a modern build automation tool designed to be highly flexible, fast, and scalable. It is widely used in Java projects, Android development, and many multi-language projects. Here's what makes Gradle stand out:

- **Flexible Build Scripts:** Gradle uses a Domain Specific Language (DSL) based on either Groovy (by default) or Kotlin. This provides a more dynamic and expressive way to define build logic compared to static XML configurations (as used in Maven).
- **Incremental Builds:** Gradle optimizes build times by determining what parts of the project have changed and rebuilding only those parts.
- **Task Automation:** Everything in Gradle is treated as a task, allowing you to create custom tasks or reuse existing ones for compiling code, running tests, packaging, and more.
- **Dependency Management:** Like Maven, Gradle can automatically download and manage dependencies from remote repositories (e.g., Maven Central, JCenter, or custom repositories).

2. Setting Up a Gradle Project

Step-by-Step Instructions

Step 1: Ensure Gradle is Installed

Before starting, verify that Gradle is installed on your Ubuntu system. Open a terminal and run:

```
gradle -v
```

If you see version information, you're set. If not, follow the [Gradle installation instructions](#) from Experiment 1.

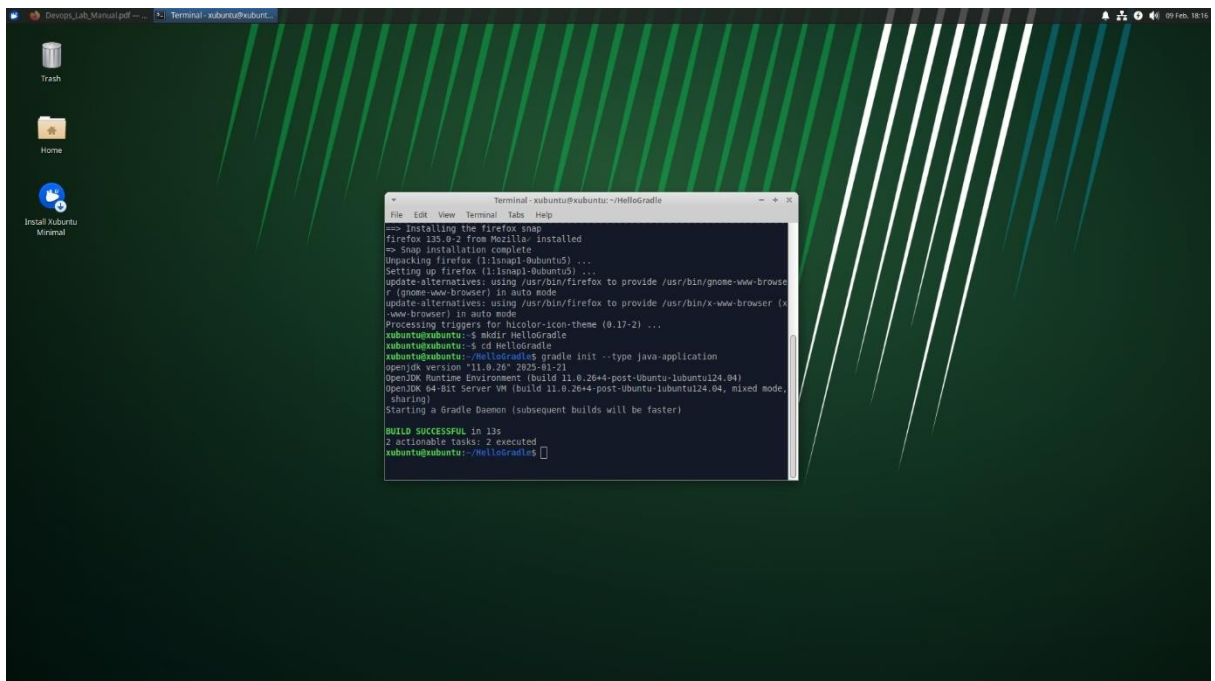
Step 2: Create a New Gradle Project

Gradle offers an interactive command to generate a new project using an initialization script.

To create a basic Java application project:

1. **Create a new directory for your project and navigate to it:**
2. `mkdir HelloGradle`
3. `cd HelloGradle`
4. **Run the Gradle init command:**
5. `gradle init --type java-application`

You will be prompted to choose between a few options (if you don't use the non-interactive mode). Choose defaults or specify details as needed.

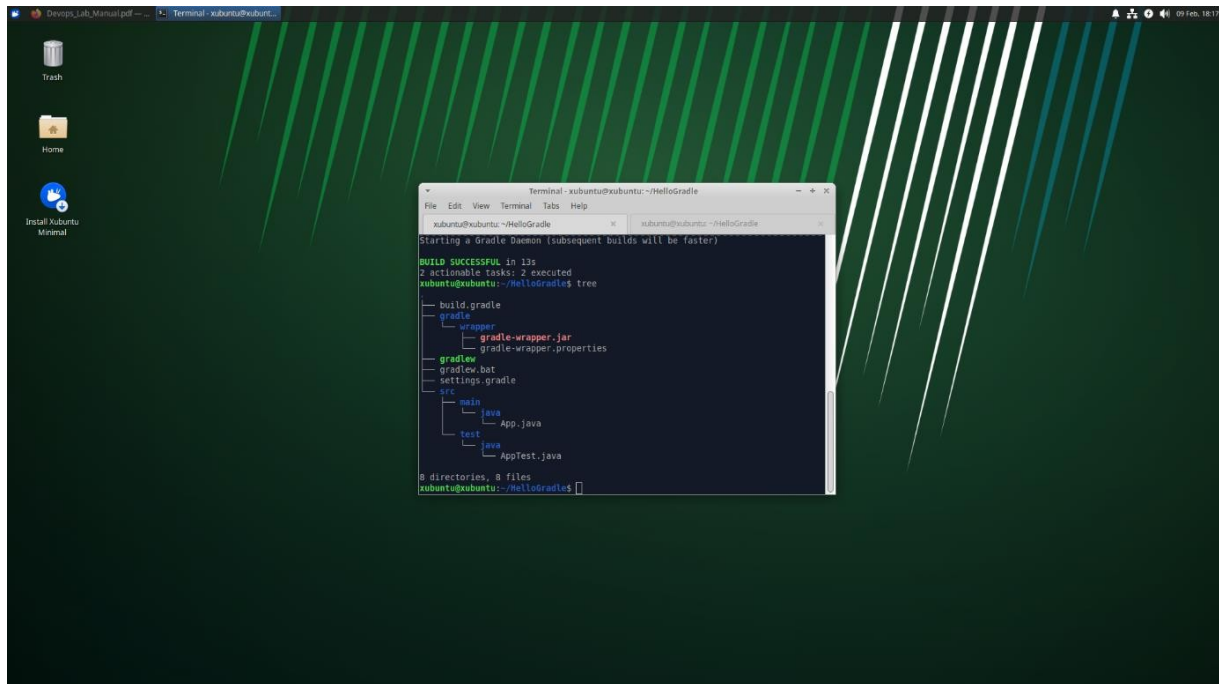


Step 3: Explore the Project Structure

After project creation, the directory structure typically looks like this:

```
HelloGradle/
├── build.gradle           // The primary build script (Groovy DSL by
                           default)
├── gradle/               // Contains Gradle wrapper files (if generated)
├── gradlew               // Unix shell script to run Gradle wrapper
├── gradlew.bat           // Windows batch script for Gradle wrapper
├── settings.gradle       // Contains project settings and names
└── src
```

```
|— main
|   └─ java
|       └─ App.java    // Your main application source file
└─ test
    └─ java
        └─ AppTest.java // Your test cases
```



Explanation of Components:

- **build.gradle:**
This is the main build script written in Groovy (or Kotlin if you choose). It defines plugins, repositories, dependencies, and tasks.
- **settings.gradle:**
A small script that defines the project's name and, in multi-project builds, the included subprojects.
- **gradlew** / **gradlew.bat:**
The Gradle wrapper scripts. They allow you to run Gradle without requiring a separate installation on every machine by automatically downloading the correct Gradle version.
- **src/main/java:**
Contains your application's source code.
- **src/test/java:**
Contains your unit tests.

3. Understanding Gradle Build Scripts

A **build script** in Gradle is a programmatic file that instructs Gradle on how to build your project. It can be written in two main DSLs:

- **Groovy DSL (`build.gradle`):** The traditional and most common syntax.
- **Kotlin DSL (`build.gradle.kts`):** A statically-typed alternative that leverages Kotlin's language features.

A. Groovy DSL Example (`build.gradle`)

Below is an example of a basic `build.gradle` for a Java application using Groovy DSL:

```
plugins {  
    // Apply the Java plugin for compiling Java code  
    id 'java'  
    // Apply the application plugin to add support for building an  
    application  
    id 'application'  
}  
  
group = 'com.example'  
version = '1.0'  
  
repositories {  
    // Use Maven Central for resolving dependencies.  
    mavenCentral()  
}  
  
dependencies {  
    // Define your dependencies. For example, JUnit for testing:  
    testImplementation 'junit:junit:4.13.2'  
}  
  
application {  
    // Define the main class for the application.  
    mainClass = 'com.example.App'  
}
```

```
// A custom task example: printing a greeting
task hello {
    doLast {
        println 'Hello, Gradle!'
    }
}
```

Explanation:

- **plugins block:** Declares the plugins used. The `java` plugin adds Java compilation and testing tasks, while the `application` plugin adds tasks for running the application.
- **group/version:** Sets project coordinates.
- **repositories block:** Configures the repository (Maven Central) where dependencies will be resolved.
- **dependencies block:** Lists the libraries your project depends on.
- **application block:** Specifies the main class of your application.
- **Custom task:** Defines a task named `hello` that prints a greeting when run.

B. Kotlin DSL Example (build.gradle.kts)

Here's how the same build configuration might look in Kotlin DSL:

Create a file named `build.gradle.kts` (or convert your file) with the following content:

```
plugins {
    // Apply the Java plugin for compiling Java code
    java
    // Apply the application plugin to add support for building an
    application
}

group = "com.example"
version = "1.0"

repositories {
    mavenCentral()
}
```

```
dependencies {  
    // Define dependencies using Kotlin DSL syntax  
    testImplementation("junit:junit:4.13.2")  
}  
  
application {  
    // Set the main class for the application  
    mainClass.set("com.example.App")  
}  
  
// A custom task example using Kotlin DSL  
tasks.register("hello") {  
    doLast {  
        println("Hello, Gradle with Kotlin DSL!")  
    }  
}
```

Explanation:

- **Kotlin DSL Syntax:** Uses a statically typed syntax, which can be more intuitive for developers familiar with Kotlin.
- **Plugins, Repositories, and Dependencies:** Defined similarly to the Groovy DSL but with Kotlin-style function calls and property access.
- **Custom Task Registration:** Uses `tasks.register` to define a task, similar in functionality to the Groovy DSL.

4. Dependency Management in Gradle

How It Works:

- **Repositories:** Gradle downloads dependencies from defined repositories (e.g., Maven Central).
 - **Dependency Notation:** Dependencies are defined in a simple format:
`group:artifact:version.`
 - **Configuration Types:** Gradle provides various configurations such as `implementation`, `compileOnly`, `runtimeOnly`, and `testImplementation` to manage the scope of each dependency.
-

Example:

In our build script examples, we included JUnit for testing:

```
dependencies {  
    testImplementation 'junit:junit:4.13.2'  
}
```

This line tells Gradle to download JUnit version 4.13.2 from Maven Central and include it in the test classpath.

Additional Dependency Configurations:

- **implementation:** Used for dependencies required at compile time.
- **runtimeOnly:** Used for dependencies required at runtime but not needed for compilation.

5. Task Automation in Gradle

In Gradle, nearly everything is a **task**. Tasks represent individual units of work (compiling code, running tests, packaging applications). Gradle comes with many built-in tasks (provided by plugins) and allows you to define your own custom tasks.

Built-in Tasks:

- **compileJava:** Compiles the source code in `src/main/java`.
- **test:** Runs tests in `src/test/java`.
- **jar:** Packages compiled code into a JAR file.
- **run:** Runs the application (if the application plugin is applied).

Creating Custom Tasks:

Custom Task in Groovy DSL:

Ensure your `build.gradle` includes the following custom task:

```
groovy  
CopyEdit  
task hello {  
  
}
```

```
doLast {  
    println 'Hello, Gradle!'  
}  
}
```

- **Run the Custom Task:**

- **Command:**

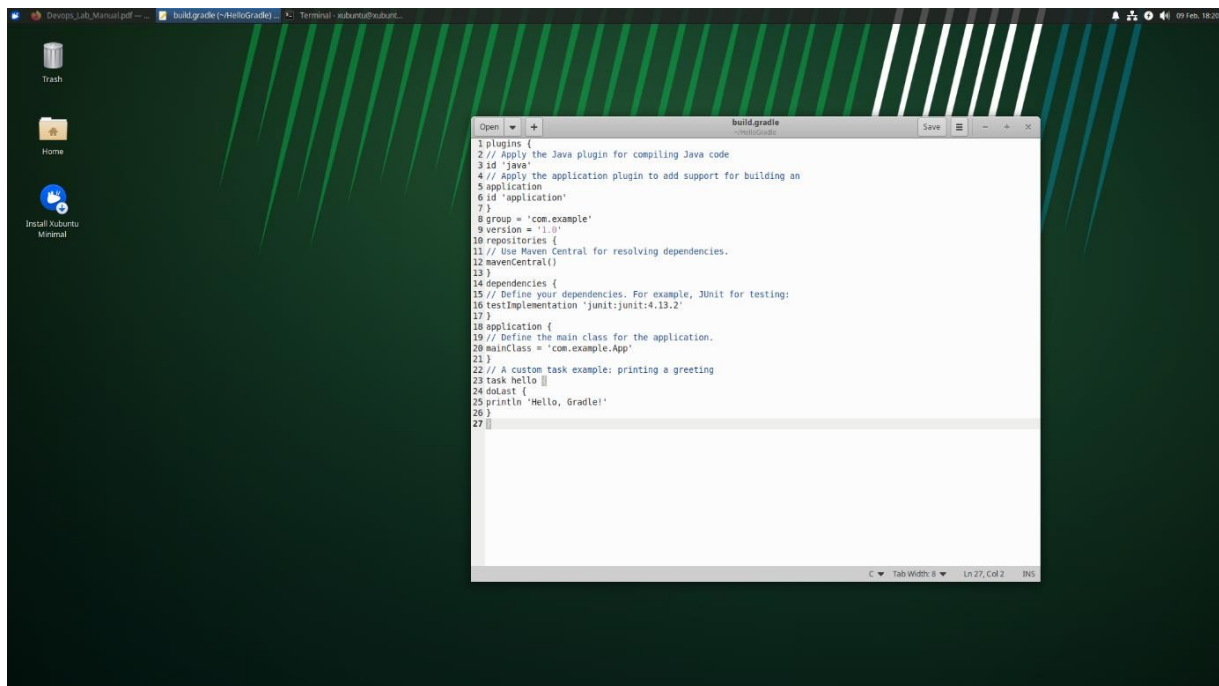
- `gradle hello`

- **Expected Output:**

You should see:

> Task :hello

Hello, Gradle!



- **Custom Task in Kotlin DSL (if using build.gradle.kts):**

Your Kotlin DSL file should include:

```
tasks.register("hello") {  
    doLast {  
        println("Hello, Gradle with Kotlin DSL!")  
    }  
}
```

- **Run the Custom Task:**

- **Command:**

```
./gradlew hello
```

- **Expected Output:**

You should see the greeting printed from the custom task.

Task Dependencies:

You can make one task depend on another. For instance, you might want your custom task to run after the `build` task:

Groovy DSL:

```
task greet(dependsOn: build) {  
    doLast {  
        println 'Build is complete! Time to celebrate!'  
    }  
}
```

Kotlin DSL:

```
tasks.register("greet") {  
    dependsOn("build")  
    doLast {  
        println("Build is complete! Time to celebrate!")  
    }  
}
```

6. Running and Verifying Your Gradle Project

Common Gradle Commands:

Build the Project:

Compile, Test, and Package:

```
gradle build
```

- **What it does:**

- o Compiles source code (`compileJava`).
- o Runs tests (`test`).
- o Packages the application into a JAR file (`jar`).

- **Expected Output:**

Look for a "BUILD SUCCESSFUL" message.

- **Screenshot Tip:** Capture the full output of the `gradle build` command

Running the Application

Run the Application:

Command:

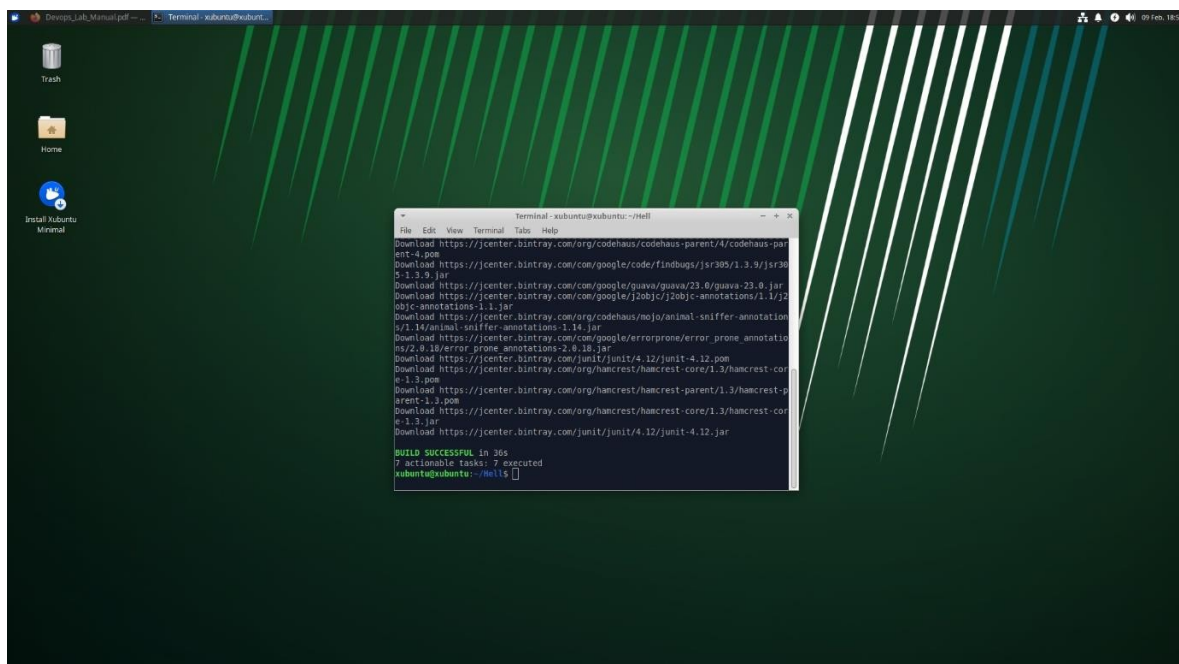
```
gradle run
```

What it does:

Runs your main class as specified in the application block.

Expected Output:

Any output from your application (for example, if your `App.java` prints a message).



Executing Custom Tasks

Run the Custom Task:

Command:

```
gradle hello
```

Expected Output:

The custom greeting message you defined earlier.

Experiment 6: Install and explore Selenium for automated testing.

What is Selenium?

Selenium is an open-source tool used for **automated testing of web applications** across different browsers and platforms. It simulates real user interactions with web pages like clicking buttons, entering text, navigating pages, etc.

Features:

- Supports multiple languages (Python, Java, C#, etc.)
- Supports multiple browsers (Chrome, Firefox, Edge, etc.)
- Useful for **regression testing**, **cross-browser testing**, and **end-to-end testing**

Prerequisites

Before using Selenium, ensure the following are installed:

1. **Python** (3.6 or above)
2. **Visual Studio Code**
3. **Google Chrome** (or your preferred browser)
4. **pip** (Python package manager)

Step-by-Step: Install and Explore Selenium on Windows Using VS Code

Step 1: Install Python

- Download and install from: <https://www.python.org/downloads>
- During installation, check the box “**Add Python to PATH**”
- To verify, open Command Prompt:
- `python --version`

Step 2: Install VS Code

- Download from: <https://code.visualstudio.com>
- Install the **Python extension** from the Extensions tab in VS Code.

Step 3: Install Selenium

Open **Command Prompt** or **VS Code terminal** and run:

```
pip install selenium
```

Step 4: Download ChromeDriver

- Go to: <https://sites.google.com/chromium.org/driver/>
- Download the ChromeDriver version that matches your Chrome browser.
- Extract the file and place it in a known directory (e.g., C:\chromedriver).

Step 5: Create a Python Project in VS Code

1. Open **VS Code**
2. Create a new folder (e.g., SeleniumTest)
3. Inside the folder, create a file: test_google.py
4. Paste the following code:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
import time

# Set path to chromedriver
driver = webdriver.Chrome(executable_path="C:\\chromedriver\\chromedriver.exe")

# Open Google
driver.get("https://www.google.com")

# Find the search box and type a query
search_box = driver.find_element(By.NAME, "q")
search_box.send_keys("Selenium with Python")
search_box.send_keys(Keys.RETURN)

# Wait for a few seconds
time.sleep(3)

# Close the browser
driver.quit()
```

Make sure to replace "C:\\chromedriver\\chromedriver.exe" with the actual path to your ChromeDriver.

Step 6: Run the Script

In the terminal inside VS Code, run:

```
python test_google.py
```

This will:

- Launch Chrome
- Open Google
- Search "Selenium with Python"
- Display results
- Close the browser after 3 seconds

Experiment 7: Develop a simple containerized application using Docker

What is Docker?

Docker is a tool designed to make it easier to **create, deploy, and run applications** using containers. Containers allow a developer to package up an application with all parts it needs (code, runtime, system tools, libraries, etc.), so it runs reliably on any environment.

What is Docker Desktop?

Docker Desktop is an easy-to-install application for **Windows** and **macOS** that enables you to:

- Build and share containerized applications
- Use a Linux-based Docker engine on Windows
- Integrate with VS Code, GitHub, Kubernetes, and WSL (Windows Subsystem for Linux)

Requirements:

- **Windows 10 Pro/Enterprise** or **Windows 11**
- **Hyper-V** or **WSL2** (enabled during installation)

Step-by-Step: Develop a Simple Containerized App Using Docker on Windows

We'll create a simple **Python Flask web app** and containerize it using Docker.

Step 1: Install Docker Desktop

1. Download Docker Desktop: <https://www.docker.com/products/docker-desktop>
2. Run the installer and follow the instructions
3. Enable **WSL2** or **Hyper-V** when prompted
4. Restart your system
5. Verify installation:
6. `docker --version`

Step 2: Create a Simple Python Flask App

1. Create a folder:

```
2. mkdir docker-flask-app
```

```
3. cd docker-flask-app
```

4. Create a file named `app.py`:

```
5. from flask import Flask
```

```
6. app = Flask(__name__)
```

```
7. @app.route("/")
```

```
8. def hello():
```

```
9.     return "Hello from Docker!"
```

```
10. if __name__ == "__main__":
```

```
11.     app.run(host="0.0.0.0", port=5000)
```

Create a `requirements.txt` file:

```
Flask = 3.1.0
```

Step 3: Write a Dockerfile

Create a file named `Dockerfile` (no extension) in the same folder:

```
# Use an official Python base image
```

```
FROM python:3.9
```

```
# Set working directory
```

```
WORKDIR /app
```

```
# Copy files
```

```
COPY requirements.txt .
```

```
RUN pip install -r requirements.txt
```

```
COPY . .
```

```
# Expose port
```

```
EXPOSE 5000
```

```
# Run the app
```

```
CMD ["python", "app.py"]
```

Step 4: Build and Run the Docker Container

Open **Terminal** or **PowerShell** inside your project directory and run:

```
docker build -t flask-docker-app .
```

Then run the container:

```
docker run -p 5000:5000 flask-docker-app
```

Now open a browser and visit:

```
http://localhost:5000
```

You should see:

```
Hello from Docker!
```

Step 5: Verify Docker Operations

Run the following commands to explore Docker:

Task	Command
List images	<code>docker images</code>
List containers	<code>docker ps -a</code>
Stop a container	<code>docker stop <container_id></code>
Remove a container	<code>docker rm <container_id></code>
Remove an image	<code>docker rmi flask-docker-app</code>

Component	Purpose
Dockerfile	Instructions to build image
Image	Packaged application
Container	Running instance of image
Docker Desktop	GUI and engine to manage containers on Windows

Experiment 8: Automate the process of running containerized application using Kubernetes

What is Kubernetes?

Kubernetes (also known as **K8s**) is an open-source container orchestration platform. It helps automate the deployment, scaling, management, and operation of containerized applications. Originally developed by Google, it is now maintained by the Cloud Native Computing Foundation (CNCF).

Why Use Kubernetes?

Running one or two containers manually is simple. But in production, you may need to run hundreds or thousands of containers across multiple servers. Kubernetes handles:

- **Automatic deployment & rollback**
- **Scaling (up/down) containers**
- **Load balancing**
- **Health monitoring**
- **Self-healing** (restart failed containers)
- **Service discovery**

Key Components of Kubernetes

Component	Description
Pod	The smallest deployable unit. A group of one or more containers.
Node	A worker machine (physical or virtual) where pods are deployed.
Cluster	A set of nodes managed by the Kubernetes control plane.
Deployment	Manages replicas of pods, handles updates, and rollbacks.
Service	Exposes a set of pods to the network (internally or externally).
Namespace	Logical separation for multiple environments or teams within a cluster.
Ingress	Manages external HTTP/HTTPS access to services inside the cluster.
ConfigMap/Secret	Manage application configuration separately from the container image.

How Kubernetes Works?

1. You define a desired state in YAML/JSON
2. You apply it using `kubectl apply -f deployment.yaml`.
3. Kubernetes Scheduler places the pods on suitable nodes.
4. Kubelet on each node ensures the containers are running as expected.
5. Controller Manager ensures the cluster always matches the desired state.

Kubernetes vs Docker

- **Docker** is for packaging applications into containers.
- **Kubernetes** is for running and managing those containers at scale.

Kubernetes in DevOps

Kubernetes is often used in DevOps pipelines for:

- CI/CD deployment
- Micro services orchestration
- Blue-green deployments
- Canary testing

Installing Kubectl

Step 1:

First go to google chrome and type kubertenes download for windows. Then select **kubertenes.io>releases>download**

Step 2:

Click on install kubectl on windows

If you have curl installed, use this command:

```
curl.exe -LO "https://dl.k8s.io/release/v1.33.0/bin/windows/amd64/kubectl.exe"
```

Step 3:

Validate the binary (optional)

Download the kubectl checksum file:

```
curl.exe -LO "https://dl.k8s.io/v1.33.0/bin/windows/amd64/kubectl.exe.sha256"
```

Step 4:

Validate the kubectl binary against the checksum file:

Using Command Prompt to manually compare CertUtil's output to the checksum file downloaded:

```
CertUtil -hashfile kubectl.exe SHA256  
type kubectl.exe.sha256
```

Using PowerShell to automate the verification using the -eq operator to get a True or False result:

```
$(Get-FileHash -Algorithm SHA256 .\kubectl.exe).Hash -eq $(Get-Content .\k
```

Step 5:

Append or prepend the kubectl binary folder to your PATH environment variable to ensure the version of kubectl is the same as downloaded:

```
kubectl version --client
```

Or use this for detailed view of version:

```
kubectl version --client --output=yaml
```

Step 6:

Install minikube by visiting <https://minikube.sigs.k8s.io/docs/start>

Download and run the installer for the latest release.

Or if using PowerShell, use this command:

```
New-Item -Path 'c:\' -Name 'minikube' -ItemType Directory -Force  
$ProgressPreference = 'SilentlyContinue'; Invoke-WebRequest -OutFile  
'c:\minikube\minikube.exe' -Uri  
'https://github.com/kubernetes/minikube/releases/latest/download/minikube-windows-  
amd64.exe' -UseBasicParsing
```

Step 7:

Add the minikube.exe binary to your PATH. Make sure to run PowerShell as Administrator.

```
$oldPath = [Environment]::GetEnvironmentVariable('Path',  
[EnvironmentVariableTarget]::Machine)
```

```
if ($oldPath.Split(';') -notcontains 'C:\minikube'){  
    [Environment]::SetEnvironmentVariable('Path', $('{0};C:\minikube' -f $oldPath),  
    [EnvironmentVariableTarget]::Machine)  
}
```

Step 8:

Start your cluster

From a terminal with administrator access (but not logged in as root), run:

```
minikube start
```

Step 9:

Interact with your cluster

If you already have kubectl installed (see documentation), you can now use it to access your shiny new cluster:

```
kubectl get po -A
```

Alternatively, minikube can download the appropriate version of kubectl and you should be able to use it like this:

```
minikube kubectl -- get po -A
```

Step 10:

Deploy applications

Create a sample deployment and expose it on port 8080:

```
kubectl create deployment hello-minikube --image=kicbase/echo-server:1.0  
kubectl expose deployment hello-minikube --type=NodePort --port=8080
```

Step 11:

It may take a moment, but your deployment will soon show up when you run:

```
kubectl get services hello-minikube
```

The easiest way to access this service is to let minikube launch a web browser for you:

minikube service hello-minikube

Alternatively, use kubectl to forward the port:

kubectl port-forward service/hello-minikube 7080:8080

Your application is now available at <http://localhost:7080/>. You should be able to see the request metadata in the application output. Try changing the path of the request and observe the changes. Similarly, you can do a POST request and observe the body show up in the output.

Experiment 9: Implementing Continuous Security with Snyk

What is Snyk?

Snyk is a developer-first security tool that helps you find and fix vulnerabilities in your application code, open-source dependencies, container images, and infrastructure as code (IaC). It integrates directly into development environments like VS Code, GitHub, GitLab, Bitbucket, Jenkins, and more.

Key Features of Snyk

Feature	Description
Snyk Open Source	Scans for known vulnerabilities in open-source libraries (e.g., npm, pip, Maven).
Snyk Code	Performs static code analysis to detect security issues in your own source code.
Snyk Container	Scans container images for OS-level vulnerabilities.
Snyk IaC (Infrastructure as Code)	Checks IaC files (Terraform, Kubernetes, etc.) for misconfigurations.
Snyk CLI	Command-line tool for scanning projects locally and in CI/CD pipelines.

What Can Snyk Scan?

Area	Examples
Application Code	JavaScript, Python, Java, C#, etc.
Dependencies	package.json, requirements.txt, pom.xml, etc.
Docker Images	Images built with Dockerfile, deployed on Kubernetes
IaC Files	Terraform (.tf), Kubernetes YAML, Helm charts

How Snyk Works

1. You install Snyk in your environment (CLI, VS Code, GitHub, etc.).
2. Snyk scans your code, dependencies, or container image.
3. It checks against its security vulnerability database.

4. Findings are shown with severity levels and suggested fixes.
5. You can choose to automatically fix or update dependencies safely.

Why Use Snyk?

- **Developer-friendly:** Easy to install and use.
- **Real-time feedback:** Spot issues while coding.
- **Open-source focused:** One of the best tools for dependency scanning.
- **Continuous monitoring:** Snyk keeps watching your codebase and alerts you if new vulnerabilities are found.

Installing Snyk using VScode

1. Open Visual Studio Code

Launch your VS Code editor on your Windows machine.

2. Go to Extensions

Click the **Extensions** icon on the sidebar, or press `Ctrl+Shift+X`.

3. Search and Install Snyk Extension

- In the search bar, type "**Snyk Security — Code, Open Source Dependencies, IaC Configurations**".
- Locate the official Snyk extension and click **Install**.

4. Trust Workspace & Authenticate

- After installation, VS Code will prompt you to **Trust Workspace and Connect**.
- Click that it will open your browser to log in to Snyk.
- Authenticate with your Snyk account (or create one for free).

5. Enable Snyk Code Scanning

- download vulnerable project form the given link <https://github.com/purpledobie/pygoat>

- Back in VS Code, you'll get a prompt: **Enable Snyk Code and start analysing**.
- Click it and in the browser, toggle **Snyk Code** to **Enabled**, then click **Save Changes**.

6. Wait for the CLI to Download

The extension automatically downloads the Snyk CLI in the background. Wait until installation completes.

7. Let Snyk Analyze Your Project

- **Automatically** on file save or project load, Snyk will run analysis.
- To **manually trigger** a scan:
 1. Click the Snyk icon in the Activity bar.
 2. Hit the **Rescan** button in the Snyk panel.

8. Review Scan Results

- Vulnerabilities show inline in your code and in the Snyk panel.
- Issues are marked with severity labels like **Medium (M)**, with suggested fixes.

Experiment 10: Working with Maven: Creating a Maven Project, Understanding the POM File, Dependency Management and Plugins

1. Introduction to Maven

Maven is a powerful build automation and project management tool primarily used for Java projects. It simplifies the build process by:

- Enforcing a **standard project structure** (convention over configuration).
- Managing **dependencies** automatically by downloading them from remote repositories (e.g., Maven Central).
- Defining a clear **build lifecycle** (compile, test, package, install, deploy).
- Allowing the integration of various **plugins** to extend functionality (e.g., testing, reporting).

2. Creating a Maven Project

Step-by-Step Process

Step 1: Open Your Terminal

Make sure you have Maven installed (refer to Experiment 1). Open your terminal on your Ubuntu system.

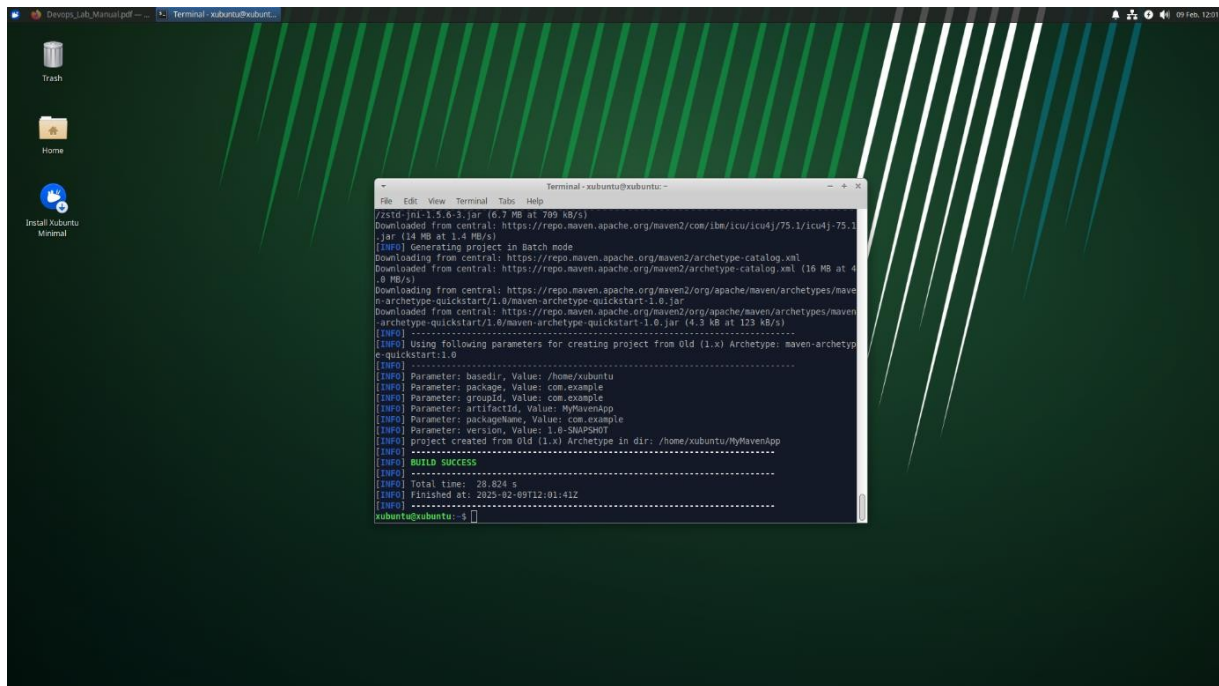
Step 2: Use Maven Archetype to Generate a New Project

Maven comes with a set of archetypes that provide you with a standard project template. Use the following command to create a new Maven project:

```
mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenApp -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

- **groupId**: Uniquely identifies your project's group (like a package name).
- **artifactId**: The name of your project (the resulting artifact).

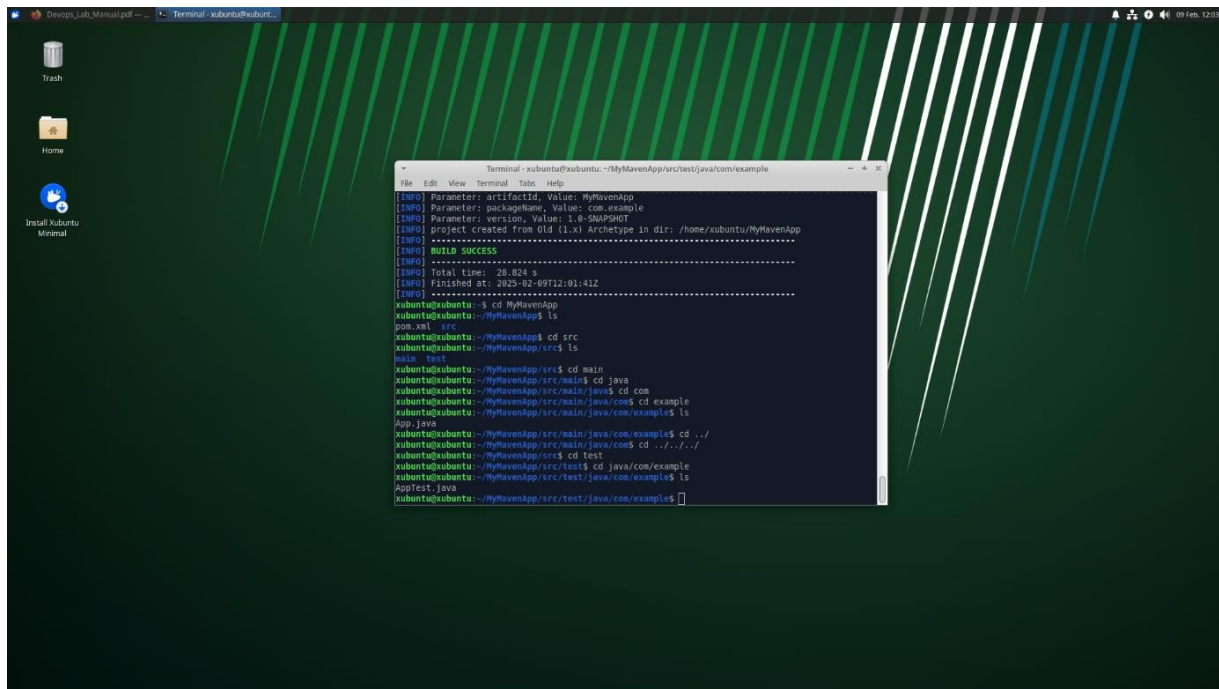
- **maven-archetype-quickstart:** A simple archetype that sets up a basic Java project with a sample unit test.
- **-DinteractiveMode=false:** Runs the command in non-interactive mode, using the provided parameters.



Step 3: Navigate to Your Project Directory

Once the command completes successfully, change your directory to the newly created project:

```
cd MyMavenApp
```



3. Maven Project Layout and Components

After generating the project, you will notice the following standard Maven directory structure:

```

MyMavenApp/
├── pom.xml
└── src
    ├── main
    │   ├── java
    │   │   ├── com
    │   │   │   ├── example
    │   │   │   │   └── App.java
    └── test
        ├── java
        │   ├── com
        │   │   ├── example
        │   │   │   └── AppTest.java

```

Explanation of Key Components

- **pom.xml:**

The **Project Object Model (POM)** file is the core of any Maven project. It contains configuration details such as project coordinates (groupId, artifactId, version), dependencies, plugins, and build settings.

- **src/main/java:**

This directory holds the source code of your application. In our example, the package structure `com.example` is created, and you have an `App.java` file.

- **src/test/java:**

This directory is for your test cases. The default example includes a basic test class, `AppTest.java`.

4. Understanding the POM File (pom.xml)

The `pom.xml` file is written in XML and is essential to how Maven operates. It includes several key sections:

Basic Structure of a POM File

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
          xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
          xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                              http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <!-- Project Coordinates -->
  <groupId>com.example</groupId>
  <artifactId>MyMavenApp</artifactId>
  <version>1.0-SNAPSHOT</version>

  <!-- Properties: Customize Java version or plugin versions -->
  <properties>
    <maven.compiler.source>11</maven.compiler.source>
    <maven.compiler.target>11</maven.compiler.target>
  </properties>

  <!-- Dependencies -->
  <dependencies>
    <!-- Example: JUnit dependency for testing -->
```

```
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.13.2</version>
  <scope>test</scope>
</dependency>
</dependencies>

<!-- Build: Configuring plugins and build settings -->
<build>
  <plugins>
    <!-- Example: Maven Compiler Plugin to compile Java code -->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.8.1</version>
      <configuration>
        <source>1.6</source>
        <target>1.6</target>
      </configuration>
    </plugin>

    <!-- Example: Maven Surefire Plugin to run tests -->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>2.22.2</version>
    </plugin>
  </plugins>
</build>
</project>
```

Key Elements Explained

- **Project Coordinates:**
 - `<groupId>`: Acts like a namespace, usually following the reverse domain name convention.
 - `<artifactId>`: The name of the project.
 - `<version>`: The current version of the project.
- **Properties:**

- Used to define values that can be referenced elsewhere in the POM (e.g., Java source and target versions).
- **Dependencies:**
 - **Dependency Management:** Maven downloads and manages external libraries. In the example, JUnit is added for testing.
 - **Scope:** Determines when a dependency is used (e.g., `compile`, `test`, `runtime`).
- **Build and Plugins:**
 - **Maven Compiler Plugin:** Ensures that your Java code is compiled with the specified Java version.
 - **Maven Surefire Plugin:** Executes unit tests during the build process.

5. Dependency Management with Maven

How Maven Manages Dependencies

- **Automatic Download:** When you specify a dependency in the `<dependencies>` section, Maven automatically downloads the library from a remote repository (usually Maven Central).
- **Transitive Dependencies:** Maven also resolves dependencies required by your dependencies.
- **Version Control:** You can specify precise versions for each dependency, ensuring consistency across builds.

Example Dependency

To add a dependency for [JUnit](#), include the following snippet in your `<dependencies>` section:

```
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.13.2</version>
  <scope>test</scope>
</dependency>
```

- **groupId, artifactId, version:** These three elements uniquely identify the dependency.

- **scope:** The `test` scope ensures that this dependency is only available during the test phase and not included in the final artifact.

6. Maven Plugins: Extending Maven Functionality

Plugins are key to Maven's flexibility, adding tasks to your build process.

Example 1: Maven Compiler Plugin

This plugin is used to compile your Java source code.

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <version>3.8.1</version>
  <configuration>
    <source>11</source>
    <target>11</target>
  </configuration>
</plugin>
```

- **Configuration:** Specifies that the project should be compiled using Java 11.

Example 2: Maven Surefire Plugin

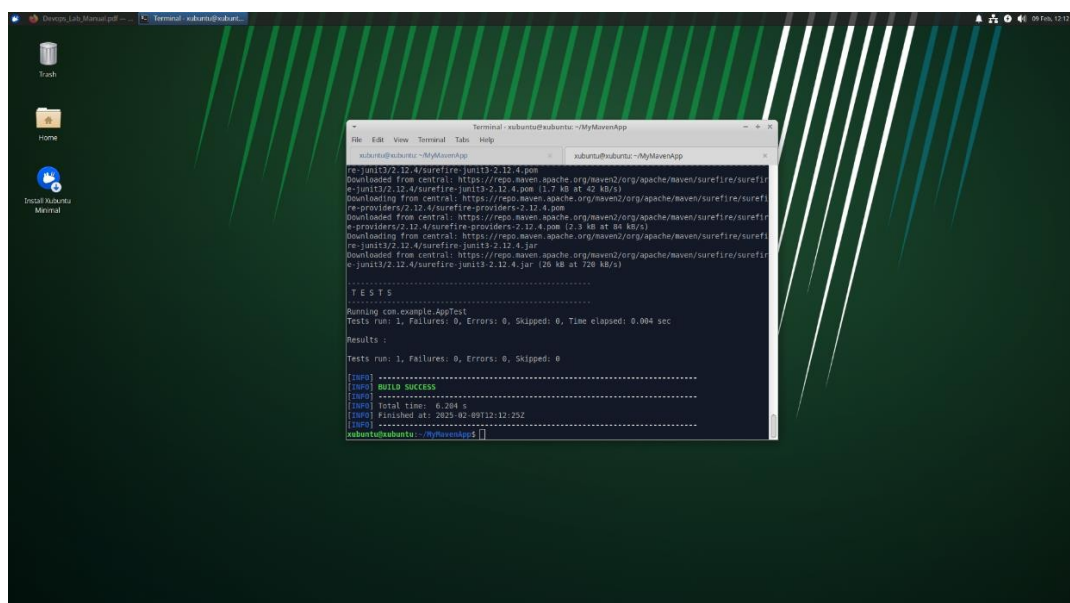
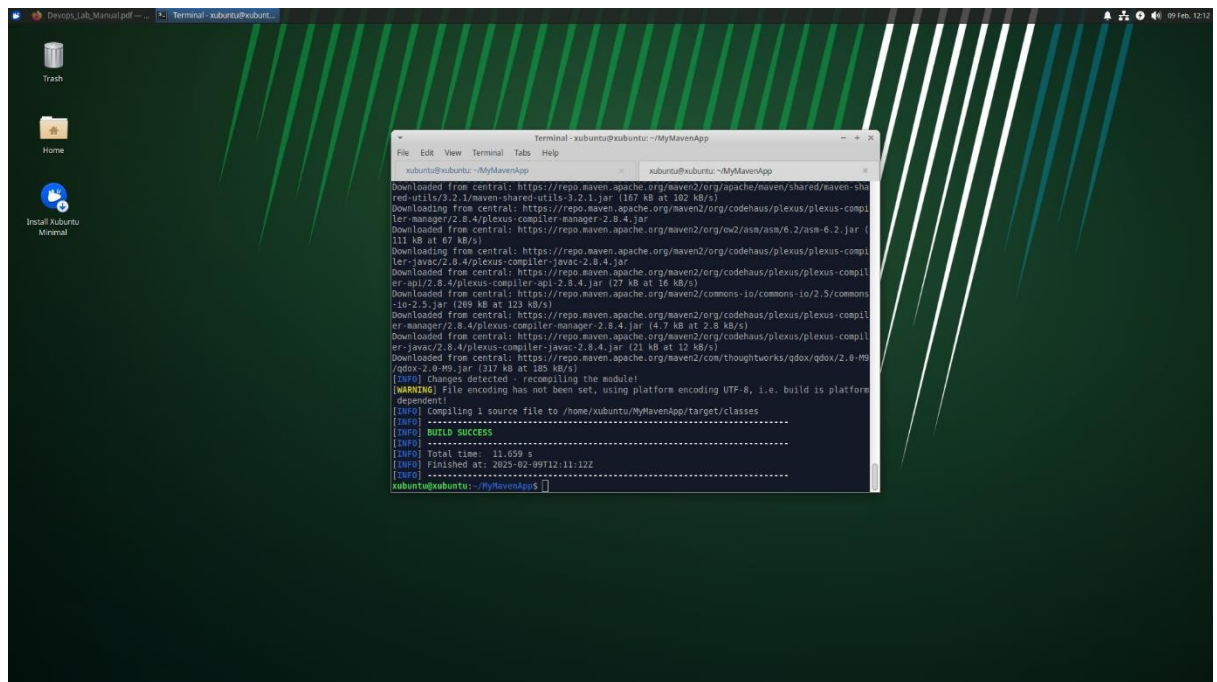
This plugin runs your unit tests during the `test` phase.

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-surefire-plugin</artifactId>
  <version>2.22.2</version>
</plugin>
```

- **Purpose:** Automatically detects and runs tests in the `src/test/java` directory.

7. Building and Testing Your Maven Project

Once your project is set up and your `pom.xml` is defined, you can use Maven commands to build and test your application.



- **Package the Application:**

```
mvn package
```

This command compiles, tests, and packages your code into a JAR file located in the `target` directory. *Screenshot Tip:* Capture the listing of the `target` directory showing the JAR file.

- **Clean the Project:**

```
mvn clean
```

This removes any files generated by previous builds.